```
// GameEventBus.cs
using System;
using System.Collections.Generic;
namespace GimmeAll.Core
{
    /// &lt;summary&gt;
    /// Typed in-process event bus. Main-thread only.
    /// Modules must communicate across assembly boundaries ONLY through this bus,
    /// the registries, or the service interfaces in Core. Never reference another module's code.
    /// &lt;/summary&gt;
    public static class GameEventBus
    {
        private static readonly Dictionary&lt;Type, List&lt;Delegate&gt;&gt; Handlers = new Dictionary&lt;Type, List&lt;
        public static void Subscribe&lt;T&gt;(Action&lt;T&gt; handler)
        {
            if (handler == null) return;
            if (!Handlers.TryGetValue(typeof(T), out var list))
            {
                list = new List&lt;Delegate&gt;(4);
                Handlers.Add(typeof(T), list);
            }
            if (!list.Contains(handler)) list.Add(handler);
        }
        public static void Unsubscribe&lt;T&gt;(Action&lt;T&gt; handler)
        {
            if (handler == null) return;
            if (Handlers.TryGetValue(typeof(T), out var list))
            {
                list.Remove(handler);
                if (list.Count == 0) Handlers.Remove(typeof(T));
            }
        }
        public static void Publish&lt;T&gt;(in T evt)
        {
            if (!Handlers.TryGetValue(typeof(T), out var list) || list.Count == 0) return;
            // Iterate defensively: handlers may unsubscribe during publish.
            for (int i = list.Count - 1; i &gt;= 0; i--)
            {
                if (i &gt;= list.Count) continue;
                if (list[i] is Action&lt;T&gt; action) action(evt);
            }
        }
        public static void ClearAll()
        {
            Handlers.Clear();
        }
    }
}
// GameEvents.cs
using UnityEngine;
```

```
namespace GimmeAll.Core
{
    // Baseline cross-module events. Ownership (publisher -&gt; consumers):
    // Levels  publishes: LevelStarted, LevelCompleted, LevelFailed, TimeRunningLow
    // Gameplay publishes: ObjectCaptured, ObjectEscaped
    // Ads     publishes: AdRewarded (from the Ads module's own namespace)
    // Core    publishes: GameStateChanged
    // Modules may define additional events in their own namespaces; the bus is generic.
    public struct LevelStarted
    {
        public int LevelId;
        public float TimeLimit;
        public int TotalTargets;
        public int GoalCount;        // -1 = clear all targets (G1)
        public string Title;        // level story title, for HUD display (W7-PLAYFIX F1; G1 same additive pattern)
        public string ChapterTitle; // chapter display name e.g.     ·      , "" = unknown (W9, same additive pattern)
        public bool HasSweepStains;  // U360-L7         /    ——           LevelRunner   ContentCatalog.ScratchKeys   spa
    }
    public struct LevelCompleted
    {
        public int LevelId;
        public float Elapsed;
        public int Cleared;
        public int Total;
        public string NextChapterTitle; //                     , UI        (W9)
        public string NextChapterName;  //               "" =   W12 G4          {  }
    }
    public struct LevelFailed
    {
        public int LevelId;
        public float Elapsed;
        public string Reason;
    }
    // Remaining time crossed the low-time line (LevelTimer.LowTimeSeconds, currently 10s).
    // Published at most once per countdown arm; re-arms if a revive pushes time back above the line.
    // Consumers: ChapterOnlooker near-defeat trembling (W7-PLAYFIX F5), future urgency cues.
    public struct TimeRunningLow
    {
        public int LevelId;
    }
    public struct ObjectCaptured
    {
        public int RecordId;
        public string TypeKey;
        public Vector2 Position;
    }
    public struct ObjectEscaped
    {
        public int RecordId;
        public string TypeKey;
```

```
    public Vector2 Position;
}
// ── U323         O-20260930-1316·R-20260930-newbie-level-01 T2/T3/T15 ──
// Gameplay SuctionField      ·Playing       FieldPressed
// SneezeStormBurst     asmdef           LevelTimer      1-5
//──                 T3         T15 LevelStarted→   FieldPressed
public struct FieldPressed
{
    public Vector2 Position;   //              /
}
public struct GameStateChanged
{
    public GameState Previous;
    public GameState Next;
}
// ── W9                    TimeRunningLow    Core      ──
// Gameplay    SneezeChargeChanged          SneezeRequested UI
//       Core                         asmdef     Core
public struct SneezeChargeChanged
{
    public float Fill01;   // 0..1
    public bool Ready;     //
}
public struct SneezeRequested
{
    //              Gameplay     Ready
}
// ── W12              SneezeChargeChanged     asmdef      ──
// Game(Bootstrap    )    Init       Menu      UI             /
// HUD     /          N     UI asmdef     Core     Levels
//                   =DEV_CONTRACTS
public struct DailyFocusResolved
{
    public int LevelId;          //   LevelStarted.LevelId    LevelData.id
    public string ChapterTitle;  //      ChapterLore  "" =
}
// ── W15-INT              SneezeChargeChanged     asmdef      ──
// Gameplay    SneezeStormBurst TryStorm      ──  SneezeRequested
//   Ready               Audio    star_release         one-shot
public struct SneezeStormBurst
{
    //                  /    Gameplay         "   "
}
// UI         CollectionScreen  Audio          UI asmdef     Audio
//         W12 DailyFocusResolved   ──
public struct CodexCryRequested
{
    public string Key;   // Resources    =Audio/Resources        /  =Audio
}
// ── W19-A       A4 UI    +A5          CodexCryRequested     asmdef      ──
```

```
// UI    ResultScreen           → Audio      +
public struct PostcardComposed
{
    //                    /       =
}
// Gameplay    ActorDudu        → Audio
public struct DuduReactionBeat
{
    public int Beat;   // 1=     ——? / 2=      ——  ——  SignatureReactionSeq
}
// UI    CollectionScreen          vh.codex.rally      → Audio
public struct CodexRallyRequested
{
    //                 profile
}
// —— W20-C              copy-runtime §6.5        DailyFocusResolved     asmdef      ——
// Game(Bootstrap InitSDK     WX.GetLaunchOptionsSync().query   lvl=N)    → UI
// CopyPools.ShareLanding      →                    id
// LevelId=1    id   LevelStarted.LevelId    0       =UI    LevelStartRequested   -1
public struct ShareLandingResolved
{
    public int LevelId;           //       query lvl=N ResultScreen.BuildSharePayload
    public string ChapterTitle;   //             ChapterLore     "" =    UI   L2
}
// —— W21-UX       UI        §   G     SneezeRequested         ——
// UI    ButtonSquash.OnPointerDown interactable    → Audio    UIClick
// Game        ——
public struct UiPressed
{
    //                    onClick
}
// —— W22-META             §    DailyFocusResolved     asmdef      ——
//   =      +1/  +5    /  /      ——Game
// UI    chip/HUD    /     Gameplay     /tint
public static class MetaItem
{
    public const int Hourglass = 0;   //      +5s    Timer.Extend
    public const int Magnet = 1;      //           ×1.25    15s
    public const int Feather = 2;     //                   =           ——X1508
    public const int Count = 3;       //
}
// Game    MetaFlow    flush/    /   /Menu     → UI         /HUD
//   Gameplay    ActorDudu       Stage=     0-3   /  /  /   §
public struct StardustChanged
{
    public int Total;   //
    public int Stage;   //      0-3
}
// Game    MetaFlow/Bootstrap      → UI    HUD
public struct ItemStockChanged
```

```
{
    public int Item;    // MetaItem.Hourglass / Magnet
    public int Count;   //
}
// UI    HUD     onClick → Game      Bootstrap   →Timer.Extend(5)/  →SuctionField.BoostRadius
//       Bootstrap
public struct ItemUsedRequested
{
    public int Item;
}
// UI    WardrobeScreen   /MetaView      → Gameplay    ActorDudu tint
//        RGB Gameplay     UI   ChapterPalette asmdef     Core
// Chapter 0=        R=G=B=-1   =
public struct OutfitEquipped
{
    public int Chapter;
    public float R, G, B;   //    tint    -1,-1,-1=
}
// ── U313           W22-META ItemUsedRequested     asmdef      ──
// Game CompanionCoordinator        CompanionChargeChanged         UI    HUD
// UI    CompanionSkillRequested       Game     ──
//     Game       MetaFlow    Ready            SneezeRequested
public struct CompanionChargeChanged
{
    public string Id;          //    id "A01".."B12"·CompanionRoster
    public float Fill01;      //      0..1 CompanionProgress.ChargeProgress01
    public bool Ready;        //
    public bool ShieldArmed;  //             ──
}
public struct CompanionSkillRequested
{
    public string Id;         //         Coordinator    Ready
}
// ── U351/panel-B R5        #3 FAIL    UpgradeCost 50/120        ──
// Game    CompanionUpgradeFlow        → UI           /
//           StardustChanged    HUD
public struct CompanionUpgraded
{
    public string Id;        //      id "A01".."B12"
    public int NewLevel;     //       2..3 Lv1
}
// ── U351/panel-B R5b      #3      ·    § .3      =      §  ──
// Game    CompanionSeatFlow        → UI                ·  3
//        Coordinator LevelStarted    ResolveSeat──
public struct CompanionSeatChanged
{
    public int Chapter;      //   1-6     ·
    public string Id;        //      id "A05".."B12"
}
// ── U351/panel-B R5b-2           UI asmdef     Core   -    =CompanionSkillRequested ──
```

```
// UI CollectionScreen             Game CompanionRosterFlow
//   →CompanionSeatFlow.TrySetSeat /   →CompanionUpgradeFlow.TryUpgrade
//    =UI                            →Game
//    UI     Game      /  /  /      ——CodexAccelSnapshotRequested
public struct CompanionSeatRequested
{
    public int Chapter;      //     1-6
    public string Id;        //      id      CompanionSeatFlow
}
public struct CompanionUpgradeRequested
{
    public string Id;        //      id      CompanionUpgradeFlow
}
public struct CompanionRosterSnapshotRequested { }
public struct CompanionRosterSnapshot
{
    public string[] Ids;       // 24 id      CompanionRoster.All
    public int[] Levels;      //      1..3      Ids
    public string[] Seats;    //         id    0-5     1-6·  =
    public int Seat2Mode;      // U351/B-R6 slice-c   ②    0=   ·1=timed·2=permanent——  /
    public int Seat2Until;    // U351/B-R6 slice-c timed       permanent=0——UI      MetaStore
    public int Sand;         //
    public int ActiveVersion; //      0=A·1=B   CompanionVersion——
    public bool[] Unlocked;   // U351/B-R6            Ids——     ·        null=
    public string TrialId;    // U351/B-R6           id  =
    public int TrialDay;     // U351/B-R6      UTC      TodayKey   =   1
    public int TodayKey;      // U351/B-R6     UTC    UI      ·  TrialDay
}
// —— U351/panel-B R6    IAA           § ·slice-a   +      ·    =slice-b ——
// UI    granted         Game CompanionAdFlow     ——         Game
// CompanionSkillRequested   -                       Flow=
// UI        =                  ·StoreScreen     Flow   =
public struct CompanionTrialRequested
{
    public string Id;        //         ·    ·   1  ——    CompanionAdFlow
}
/// &lt;summary&gt;       Game→UI                        &lt;/summary&gt;
public struct CompanionTrialArmed
{
    public string Id;        //                 ·
    public int Day;          //    UTC       1
}
public struct CompanionRechargeRequested
{
    public string Id;    // U351/B-R6 slice-c        id  =   ①   ——HUD      id
}
// —— U351/panel-B R6       ②     § .2   2         =     24h   500        ——
// UI companion_seat    granted           Game CompanionAdFlow       ——
//          Game CompanionSkillRequested   -
// Flow=            =CompanionSeatFlow.TryUnlockSeat2Permanent
```

```
//    CompanionSeat2Unlocked Game→UI    /        HUD
public struct CompanionSeatUnlockRequested { }
/// &lt;summary&gt;  ②     Game→UI  Mode 1=   24h   ·2=                &lt;/summary&gt;
public struct CompanionSeat2Unlocked
{
    public int Mode;        // 1=timed 24h   ·2=permanent 500
    public int UntilTs;    // timed         permanent=0——    ·    MetaStore
}
// —— U351/panel-B R6 slice-c        UI asmdef     Core   -    =CompanionSeatRequested ——
// UI StoreScreen   2            Game CompanionSeatFlow AutoBind
// TryUnlockSeat2Permanent                      CompanionSeat2Unlocked
//   /          ——slice-c2
public struct CompanionSeat2PurchaseRequested { }
// —— U314-P0-4             W22-META StardustChanged     asmdef      ——
//           v1     3   / =   2  /  1    /   3      →      60
//       P0-3   =                60          +20
// Game DailyTaskFlow    DailyTasksChanged   /    +Menu       UI        /
//     ——              DailyTaskCatalog   Core=UI asmdef     Game
public static class DailyTaskCatalog
{
    public const int TaskCount = 3;
    // U360-L6B1       L6 § .1  10      1  1           5
    //        =        ——     30       30&gt;3   6bit    ·DailyTaskFlow
    public static readonly int[] Targets = { 2, 30, 3 };   //   0   2 /  1   30 /  2   3
    public const int SandPerTask = 20;                    //    60 →      20 P0-3
}
public struct DailyTasksChanged
{
    public int Day;                                  // UTC    unix/86400·DailyLevelPolicy
    public int Progress0, Progress1, Progress2;   //
    public bool Claimed0, Claimed1, Claimed2;      //
}
// Game Bootstrap    TryConsume    ·ItemStockChanged     Game DailyTaskFlow
//    1           ——ItemUsedRequested
public struct ItemEffectApplied
{
    public int Item;   //           id
}
// —— X1432      UI      UI asmdef     Core   -    =ItemUsedRequested ——
// UI         Game DailyTaskFlow        ——TaskId   /  /
//       UI       =
public struct DailyTaskClaimRequested
{
    public int TaskId;
}
// UI     /     OnEnable+Open    Game DailyTaskFlow          ——
//         /          Menu        spirit
public struct DailyTaskSnapshotRequested { }
// —— U351/panel-B R7             DailyTaskCatalog     asmdef     ·      v1 §  ——
//  10/20/40…            7   +14pp          =
```

```
//
//            ·      v1 §        7   10/20/40/80/160/320/640     640
// ——             =       U332
// Game CheckInFlow    CheckInChanged    +Menu       UI
// ——D7 habit  =            ·    D2              CheckInCatalog
//   Core=UI asmdef     Game     DailyTaskCatalog
public static class CheckInCatalog
{
    public const int BaseSand = 10;        //             10/20/40/80/160/320/640
    public const int MaxLadderIndex = 6;   //     0  ·  7  =640    Ladder=
    public const int LadderDays = 7;       //       UI
    /// &lt;summary&gt;  ladder      0  ·10&amp;lt;&amp;lt;ladder                640
    ///           640——                          &lt;/summary&gt;
    public static int RewardFor(int ladder)
    {
        if (ladder &lt; 0) return BaseSand;
        if (ladder &gt; MaxLadderIndex) return BaseSand &lt;&lt; MaxLadderIndex;
        return BaseSand &lt;&lt; ladder;
    }
}
public struct CheckInChanged
{
    public int Day;             // UTC    unix/86400·DailyTaskFlow      0=
    public int Ladder;          //        0..MaxLadderIndex·
    public bool TodayClaimed;   //      CheckInDay==  ·
    public int TodayReward;     //         =       ·  =   ——Ladder
}
// UI        Game CheckInFlow        ——          UI
public struct CheckInClaimRequested { }
// UI        OnEnable+Open    Game CheckInFlow
public struct CheckInSnapshotRequested { }
// —— U314-B4             DailyTaskCatalog     asmdef     ·      v1 §  ——
//               →   5/5         →    +100   +1
//            =     CodexCatalog        ——
//
// Game ChapterChestFlow    ChapterChestChanged   /    +Menu        UI
//          X/5 /    ——        =         Core=UI asmdef     Game
public static class ChapterChestCatalog
{
    public const int ChapterCount = 18;
    public const int LevelsPerChapter = 5;
    public const int TotalLevels = 90;
    public const int SandPerChest = 100;   //        P0-3        ·
    /// &lt;summary&gt;           RNG    =  /   =  ——             &lt;/summary&gt;
    public static int ChestItemOf(int chapter) { return (chapter &amp;amp; 1) == 1 ? MetaItem.Hourglass : MetaItem.Magne
}
public struct ChapterChestChanged
{
    public int Chapter;     // 1-18
    public int LevelsDone;  //        0-5
```

```
    public bool Claimable;  // 5/5
    public bool Claimed;    //
}
// UI            Game ChapterChestFlow          ——   /   /
//                 UI       =
public struct ChapterChestClaimRequested
{
    public int Chapter;
}
// UI     OnEnable/        Game ChapterChestFlow          ——
// Chapter 0=                  =18
public struct ChapterChestSnapshotRequested
{
    public int Chapter;
}
// —— X1459           U314-B4-3·    v1 §          ·DailyTaskFlow      ——
// Game CodexFlow    CodexFullHouseChanged Granted=true=    ·           /
//       UI CollectionScreen       CodexFullHouseSnapshotRequested Game   =
//               /           ChapterChestClaimRequested
public struct CodexFullHouseChanged
{
    public bool Granted;   // true=                false=     ·
    public int Sand;       //       =CodexHouseCatalog.FullHouseSand
}
// UI          Game CodexFlow                  ——
// LevelCompleted/
public struct CodexFullHouseSnapshotRequested
{
    //
}
/// &lt;summary&gt;X1459         ChapterChestCatalog            =
///     17                    0   ≥ 80     =LevelProgressStore
///  Levels asmdef·CodexFlow   asmdef               UI   CodexCatalog 17
///        TestCodexFlow                    &lt;/summary&gt;
public static class CodexHouseCatalog
{
    public const int FriendCount = 17;          //              1-17·  UI
    public const int LevelsPerChapter = 5;      //      ChapterChestCatalog
    public const int FullHouseProgress = (FriendCount - 1) * LevelsPerChapter;   // 80   17
    public const int FullHouseSand = 500;       //          =     ·P0-3
}
// —— X1460       ·  ③             v1 § 3 Butler's Gift  ·DailyTaskCatalog      ——
//     =                1     LevelFailed    → pending           →
// AdRewarded(supply)    +1         RNG → LevelStarted(=pending)
//      DailyCap UTC     ·                 Core=UI asmdef     Game
public static class LevelSupplyCatalog
{
    public const int DailyCap = 3;                   //
    public const int TotalLevels = 90;               // ChapterChestCatalog
    /// &lt;summary&gt;           RNG    =  /   =  ——             &lt;/summary&gt;
```

```
    public static int SupplyItemOf(int levelId) { return (levelId &amp;amp; 1) == 1 ? MetaItem.Hourglass : MetaItem.Magn
}
// Game LevelSupplyFlow        /  /  /Menu     UI       /       ——  =
public struct LevelSupplyChanged
{
    public int Level;         // pending   0=
    public bool Eligible;    //     pending&amp;gt;0
    public int Item;          //         id LevelSupplyCatalog.SupplyItemOf
    public int GrantedToday;  //
    public bool CappedToday;  //
}
// UI     OnEnable+Open    Game LevelSupplyFlow          ——
//  DailyTaskSnapshotRequested   spirit
public struct LevelSupplySnapshotRequested { }
// —— X1474       ④             v1 § 4   ×        1       ·LevelSupplyCatalog      ——
//            1                    [75,80)·   =80    =+1
//               RNG   =       ·supply              UTC
//         Core=UI asmdef     Game     DailyTaskCatalog
public static class CodexAccelCatalog
{
    public const int DailyCap = 2;    //              ·supply 3/offline
    /// &amp;lt;summary&amp;gt;           RNG   =  /  =  ——   =         &amp;lt;/summary&amp;gt;
    public static int AccelItemOf(int levelId) { return (levelId &amp;amp; 1) == 1 ? MetaItem.Hourglass : MetaItem.Magne
}
// Game CodexAccelFlow      /    /Menu        UI          ——  =
public struct CodexAccelChanged
{
    public int Missing;        //      N   0=
    public bool Eligible;      //        1
    public int Item;           //         id CodexAccelCatalog.AccelItemOf
    public int GrantedToday;   //
    public bool CappedToday;   //
}
// UI     OnEnable+Open    Game CodexAccelFlow          ——
public struct CodexAccelSnapshotRequested { }
// —— X1471       ⑤           v1 § 5                      ·LevelSupplyCatalog      ——
//   ≥MinAwaySeconds       SandPerHour/            ·MaxCapHours
// Boot   Menu        pending/away prefs →         base /       AdRewarded
// ("offline"      )   ×2              away       ·Play
//      Core=UI asmdef     Game     DailyTaskCatalog
public static class OfflineEarningsCatalog
{
    public const int SandPerHour = 8;           //           60/   100     ·
    public const int MaxCapHours = 12;           //           ——96
    public const int MinAwaySeconds = 1800;     //        30    ——
}
// Game OfflineEarningsFlow        /    /Menu       UI          ——  =
public struct OfflineEarningsChanged
{
    public int Pending;        //      0=
```

```
    public int AwaySeconds;   //         ·     N
}
// UI            Game OfflineEarningsFlow    base     ——
//   pending           ·DailyTaskClaimRequested
public struct OfflineEarningsClaimRequested { }
// UI    OnEnable    Game                 ——LevelSupplySnapshotRequested   spirit
public struct OfflineEarningsSnapshotRequested { }
// —— X1508          B   slice-d·ItemUsedRequested     asmdef      ——
// UI           StorePurchaseRequested → Game StoreFlow      TrySpendSand    +
// GrantItem    +    StardustChanged/ItemStockChanged MetaFlow         → UI
// StorePurchaseSettled           Ok=false             /    =
//       U280      ·  =       200×2+150×3+300=1,150      /
//   =     Ads           ·               Core=UI asmdef
// Game     DailyTaskCatalog
public struct StorePurchaseRequested
{
    public int OfferId;   // StoreDealCatalog.Offer_* Game          offer
}
public struct StorePurchaseSettled
{
    public int OfferId;
    public bool Ok;      // true=       false=              UI
}
// X1962 R3#9          panel-B      UI        ——
// Settled(Ok=false)      Audio         SfxFlow       UI     /
//    offer          StoreFlow
public struct StorePurchaseRejected
{
    public int OfferId;   //         StoreDealCatalog.Offer_*
}
public static class StoreDealCatalog
{
    public const int OfferBundle = 0;      //
    public const int OfferMagnet = 1;       //
    public const int OfferHourglass = 2;   //
    public const int OfferFeather = 3;      //
    public const int PriceBundle = 699;     //          1,150·  6.1
    public const int PriceMagnet = 200;
    public const int PriceHourglass = 150;
    public const int PriceFeather = 300;
    /// &lt;summary&gt;Offer       offer   0=         &lt;/summary&gt;
    public static int PriceOf(int offerId)
    {
        switch (offerId)
        {
            case OfferBundle: return PriceBundle;
            case OfferMagnet: return PriceMagnet;
            case OfferHourglass: return PriceHourglass;
            case OfferFeather: return PriceFeather;
            default: return 0;
```

```
        }
    }
    /// &lt;summary&gt;   offer →     bundle   BundleItems/BundleCounts    &lt;/summary&gt;
    public static int ItemOf(int offerId)
    {
        switch (offerId)
        {
            case OfferMagnet: return MetaItem.Magnet;
            case OfferHourglass: return MetaItem.Hourglass;
            case OfferFeather: return MetaItem.Feather;
            default: return -1;
        }
    }
    //             x2·   x3·     x1 ——
    public static readonly int[] BundleItems = { MetaItem.Magnet, MetaItem.Hourglass, MetaItem.Feather };
    public static readonly int[] BundleCounts = { 2, 3, 1 };
}
// ————————————————————————————————————————————————————————————————
// —— U360 L3/L6          O-20261008-1438·DailyTaskCatalog     asmdef      ——
// L3    Design/evidence/gimmeall-feel/L3_           V1.md § /§  +
// L6    L6_             V1.md § /§       /    /   /
//                  asmdef——    Core=
// ————————————————————————————————————————————————————————————————
// Levels WaveDirector/LevelRunner    Gameplay AmbushBeat      /Audio    sting /
// Tests       WaveIndex 0=W0         SpawnAll   —— 2
// 1=W2     2=W3     Audio   sting    ≥1 W0=
public struct WaveAdvanced
{
    public int LevelId;
    public int WaveIndex;   // 0=W0    1=W2     2=W3
}
// Levels WaveDirector    W2         L3 §        ·S3 c3a   3
//       c3b   1  =       c3c   2  =     Gameplay SneezeSystem
// Gameplay LandmarkBeacon        Game CompanionCoordinator
// Game HiddenTreasureFlow         spawn        +1  Tests      1
public struct MidEventTriggered
{
    public int LevelId;
    public int Kind;   // MidEventCatalog
}
//              Core=    Levels/    Gameplay·Game      ·DailyTaskCatalog
public static class MidEventCatalog
{
    public const int SneezeWindow = 0;        //    =     1     / 4
    public const int BuddySkillLesson = 1;   //       =      S3 c3b   · 1
    public const int HiddenTreasure = 2;     //    =     +1 S3 c3c   · 2  ·HiddenTreasureFlow
    public const int LandmarkPolishBeat = 3; //      =         S3 c3a· 3
}
// Gameplay LandmarkBeacon  3                 L3 §    3
// Tests      1/
```

```
public struct LandmarkPolished
{
    public int LevelId;
}
// Gameplay ChapterFinale  5                   L3 §    5
// Tests      1/
public struct ChapterFinaleStarted
{
    public int LevelId;
}
// Gameplay AmbushBeat  2               L3 §    2
// Audio       squeak Tests
public struct AmbushRevealed
{
    public int LevelId;
}
// Gameplay SuctionField                L6 § .1 P0   ——   ≥0.8u
//    ≤0.35s=1    Audio      swish ScratchTarget
public struct SweepBeatStruck
{
    public float X;   //
    public float Y;
}
// Core SystemUnlockGate.MarkUnlocked /Game UnlockFlow    UI
//         L6 § .1                  SystemUnlockGate
public struct SystemUnlockChanged
{
    public int Bits;
}
// —— U360-L6B3        L6    § .3·SweepBeatStruck     asmdef      ——
// Gameplay ChapterEvolveFlow     LevelStarted  ·id%5==1
//     =     ·§        buff       Tests
// c2        UI ArrivalBanner      ——  L3 §  0-1s         ·
public struct EvolveMomentTriggered
{
    public int LevelId;
    public int Chapter;
    public string EvolveName;   // c2     EvolveNameFor    ——UI   Gameplay
}
// —— U360-L6B4        L6    § .4        ·EvolveMomentTriggered     asmdef      ——
//     TimeRunningLow     &lt;80% goal          A   10   vs B
//     ·   +15   ·            1               /AdFrequencyGate
//    90s/3               level_complete never mid-level     ·WeChatAdService
//       → A        choice_time             opt-in
//     RegisterInterstitialShown      90s/3   =        Game InLevelChoiceFlow
//    Offered/Resolved UI ChoiceBanner    UI    PickRequested Game
//——  -    =CompanionSkillRequested         Core=UI asmdef     Game
public static class InLevelChoiceCatalog
{
    public const float ClearGate01 = 0.8f;             //            &lt;80% goal
```

```
    public const float ExtendSeconds = 10f;            // A     10
    public const int GambleBonusSand = 15;             // B          +15
    public const string AdPlacement = "choice_time";   // A        AdConfig    +RewardCatalog
}
// Game InLevelChoiceFlow         UI ChoiceBanner
public struct InLevelChoiceOffered
{
    public int LevelId;
    public bool ExtendAvailable;   // AdFrequencyGate      false=    /   ——A
}
// UI ChoiceBanner    onClick    Game InLevelChoiceFlow
public struct InLevelChoicePickRequested
{
    public bool TookExtend;   // true=A   10   /false=B
}
// Game          UI ChoiceBanner       Tests
public struct InLevelChoiceResolved
{
    public int LevelId;
    public bool TookExtend;
    public bool GambleArmed;   // B             +15
}
// —— U360-L6B4 c3        L6    § .2        ·InLevelChoiceCatalog     asmdef      ——
//    ≥5 ComboTracker 1.2s  ——  B5                A
// PointVacuum       3       vs B      SuctionField.BoostRadius ×1.5    5s
//——            =        ·         =      1
// ChainGeneration      +     8s Game ComboChoiceFlow    Offered/Resolved
// UI ChoiceBanner          UI    PickRequested Game
//——  -    =InLevelChoicePickRequested         Core=UI asmdef     Game
public static class ComboChoiceCatalog
{
    public const int TriggerChain = 5;        //    ≥5    ComboTracker.Chain
    public const float SilenceSec = 8f;       //      8s       8s     ——
    public const int PointCaptureCount = 3;   // A           3       PointVacuum
    public const float BoostMult = 1.5f;       // B          ×1.5      1.25   API
    public const float BoostSec = 5f;          //    5s
}
// Game ComboChoiceFlow          UI ChoiceBanner              ·
public struct ComboChoiceOffered
{
    public int LevelId;
}
// UI ChoiceBanner    onClick    Game ComboChoiceFlow
public struct ComboChoicePickRequested
{
    public bool TookPoint;   // true=A     /false=B
}
// Game      ·         UI ChoiceBanner       Tests
public struct ComboChoiceResolved
{
```

```
        public int LevelId;
        public bool TookPoint;
    }
    // ── U360-L6B5 c2           L6    § .2        ·Toon Blast  ·ComboChoiceCatalog     asmdef      ──
    //      23+ 22    HardHint     =         =SystemUnlockGate 22-30
    //        A        +5s=MetaItem.Hourglass        vs B
    // ×1.25    15s=MetaItem.Magnet        ──
    //                LevelTimer        Offered/Resolved    ──
    //              Game PreLevelChoiceFlow    Offered/Resolved UI ChoiceBanner
    //          UI    PickRequested Game     ──  -    =InLevelChoicePickRequested
    //         Core=UI/Levels asmdef     Game
    public static class PreLevelChoiceCatalog
    {
        public const int FirstOfferLevel = 23;      //      23+    § .2   22=  +
        public const float HourglassBonusSec = 5f;   // A        +5s MetaItem.Hourglass
        public const float MagnetBoostMult = 1.25f;  // B          ×1.25 MetaItem.Magnet
        public const float MagnetBoostSec = 15f;     //    15s
    }
    // Game PreLevelChoiceFlow        UI ChoiceBanner                    LevelTimer
    public struct PreLevelChoiceOffered
    {
        public int LevelId;
    }
    // UI ChoiceBanner    onClick    Game PreLevelChoiceFlow
    public struct PreLevelChoicePickRequested
    {
        public bool TookHourglass;   // true=A    /false=B
    }
    // Game      ·         UI ChoiceBanner       LevelTimer       Tests
    public struct PreLevelChoiceResolved
    {
        public int LevelId;
        public bool TookHourglass;
    }
}
// GameStateMachine.cs
namespace GimmeAll.Core
{
    public enum GameState
    {
        Boot,
        Menu,
        Playing,
        LevelComplete,
        LevelFailed,
        Paused
    }
    /// &lt;summary&gt;
    /// Global coarse-grained game state. Modules observe GameStateChanged; they must not set it
    /// (the future Bootstrap owns transitions), except the self-test entry points during development.
```

```
    /// &lt;/summary&gt;
    public static class GameStateMachine
    {
        public static GameState Current { get; private set; } = GameState.Boot;
        public static void Set(GameState next)
        {
            if (next == Current) return;
            var prev = Current;
            Current = next;
            GameEventBus.Publish(new GameStateChanged { Previous = prev, Next = next });
        }
        public static void ResetToBoot()
        {
            Set(GameState.Boot);
        }
    }
}
// Bootstrap.cs
using UnityEngine;
using GimmeAll.Core;
using GimmeAll.Gameplay;
using GimmeAll.Levels;
using GimmeAll.Social;
using GimmeAll.UI;
using GimmeAll.Ads;
using GimmeAll.Replay;
namespace GimmeAll.Game
{
    /// &lt;summary&gt;
    ///     Composition Root                          DEV_CONTRACTS
    ///                 /   /         F            Levels
    ///                /
    ///                 ——
    /// W4           ContentCatalog 30     B1   2——LevelCatalog 3
    /// &lt;/summary&gt;
    public static class Bootstrap
    {
        private const float CameraSize = 7.5f;      //      11.25×15 3:4  W10     20.5u   CameraRig
        private const float ReviveSeconds = 10f;    // revive     =  10   RewardCatalog
        private static LevelRunner _runner;
        private static LevelTimer _timer;
        private static BackdropBoard _backdrop;
        [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
        private static void Init()
        {
            EnsureInfra();
            EnsureServices();
            GameEventBus.Subscribe&lt;LevelStartRequested&gt;(OnStartRequested);
            GameEventBus.Subscribe&lt;ContinueRequested&gt;(OnContinueRequested);
            GameEventBus.Subscribe&lt;DailyLevelRequested&gt;(OnDailyRequested);
```

```
            GameEventBus.Subscribe&lt;ReviveRequested&gt;(OnReviveRequested);
            GameEventBus.Subscribe&lt;ReturnMenuRequested&gt;(OnReturnMenu);
            // W22-META        §    ——  +5s    Timer.Extend    /  =     ×1.25 15s
            //          HUD        =
            GameEventBus.Subscribe&lt;ItemUsedRequested&gt;(OnItemUsedRequested);
            //        W6-B2  LevelStarted        LevelId GameStateMachine    Set
            //
            GameEventBus.Subscribe&lt;LevelStarted&gt;(OnLevelStarted);
            // W12             ——
            //      Menu           Init
            GameEventBus.Subscribe&lt;GameStateChanged&gt;(OnGameStateChanged);
            PublishDailyFocus();
        }
        //         W4         BeforeSceneLoad        AfterSceneLoad   /
        // ①                W4-A + InitSDK——WX.*          W4-A    grep
        //   C#→wx           SDK        InitSDK    /
        // ②    PersistentLeaderboardService        W4-G
        // ③
        private static void EnsureServices()
        {
#if !UNITY_EDITOR &amp;&amp; (WEIXINMINIGAME || UNITY_WEBGL)
            ServiceRegistry.Register&lt;ILeaderboardService&gt;(new WeChatLeaderboardService());
            WeChatWASM.WX.InitSDK(code =&gt;
            {
                Debug.Log("VH|WX|InitSDK|" + code);
                ShareLandingGate.ResolveFromLaunch();   // W20-C             query §6.5
            });
            // W8          /
            // InitSDK   ——         SDK
            ServiceRegistry.Register&lt;IAdService&gt;(new WeChatAdService());
            ServiceRegistry.Register&lt;IShareService&gt;(new WeChatShareService());
            // W10-GROWTH      branch-analytics         =
            ServiceRegistry.Register&lt;IAnalyticsService&gt;(new WeChatAnalyticsService());
#endif
            var inner = ServiceRegistry.Get&lt;ILeaderboardService&gt;();
            if (inner == null) inner = new StubLeaderboardService();
            if (!(inner is PersistentLeaderboardService))
                ServiceRegistry.Register&lt;ILeaderboardService&gt;(new PersistentLeaderboardService(inner));
        }
        private static void EnsureInfra()
        {
            if (Camera.main == null)
            {
                var camGo = new GameObject("[Game] Camera");
                camGo.tag = "MainCamera";   // U360-L7            MainCamera   =Camera.main      null       /      /
                var cam = camGo.AddComponent&lt;Camera&gt;();
                // AudioListener W6-A                     WebGL/
                // AudioSource    ——51+91                       Play
                camGo.AddComponent&lt;AudioListener&gt;();
                cam.orthographic = true;
```

```
    cam.orthographicSize = CameraSize;
    cam.clearFlags = CameraClearFlags.SolidColor;
    cam.backgroundColor = new Color(0.16f, 0.14f, 0.22f, 1f);
    cam.transform.position = new Vector3(0f, 0f, -10f);
}
// W10-E2       LateUpdate lerp   SuctionField.FieldPos +
if (Object.FindObjectOfType&lt;CameraRig&gt;() == null)
{
    var rigGo = new GameObject("[Game] CameraRig");
    rigGo.AddComponent&lt;CameraRig&gt;();
}
//               A
var suction = Object.FindObjectOfType&lt;SuctionField&gt;();
if (suction == null)
{
    var fieldGo = new GameObject("[Game] SuctionField");
    suction = fieldGo.AddComponent&lt;SuctionField&gt;();
    suction.Config = ScriptableObject.CreateInstance&lt;SuctionFieldConfig&gt;();
}
suction.FollowActor = true;   // W10                    W9     =
//      W6            "     "
if (Object.FindObjectOfType&lt;RevealVeil&gt;() == null)
{
    var veilGo = new GameObject("[Game] RevealVeil");
    veilGo.AddComponent&lt;RevealVeil&gt;();
}
//        W6-B2  sortingOrder -1000     (-999)  ——
// FindObjectOfType                                  Menu
//                    LevelStarted
_backdrop = Object.FindObjectOfType&lt;BackdropBoard&gt;();
if (_backdrop == null)
{
    var backdropGo = new GameObject("[Game] Backdrop");
    _backdrop = backdropGo.AddComponent&lt;BackdropBoard&gt;();
}
ApplyBackdrop(1);   // W10 scene_*       ShowLarge           bg cover-fit
//         W7-G1 ART-DIRECTION §6  L1        / L2      / L3
//              RevealVeil                 RevealVeil
//              Start    1
if (Object.FindObjectOfType&lt;AmbientLoopLayer&gt;() == null)
{
    var ambientGo = new GameObject("[Game] AmbientLoop");
    ambientGo.AddComponent&lt;AmbientLoopLayer&gt;();
}
// W10            Ensure    +   =      ApplyBackdrop
var dudu = ActorDudu.Ensure();
Vector2 spawn = ActorDudu.WalkBounds.center;
dudu.transform.position = new Vector3(spawn.x, spawn.y, 0f);
dudu.SetTarget(spawn);   //          Awake
// W10-GROWTH   +               /UI             /
```

```
    if (Object.FindObjectOfType&lt;TelemetryRelay&gt;() == null)
    {
        var tlGo = new GameObject("[Game] TelemetryRelay");
        tlGo.AddComponent&lt;TelemetryRelay&gt;();
    }
    if (Object.FindObjectOfType&lt;HapticsRelay&gt;() == null)
    {
        var hpGo = new GameObject("[Game] HapticsRelay");
        hpGo.AddComponent&lt;HapticsRelay&gt;();
    }
    // W17-NEWBIE-FLOW                  +
    if (Object.FindObjectOfType&lt;TutorialHint&gt;() == null)
    {
        // W24-PLAY-SEE        UI Graphic     Canvas      ——     GO
        //       W17         /             SafeArea
        var tutGo = new GameObject("[Game] TutorialHint");
        if (UIRoot.SafeArea != null) tutGo.transform.SetParent(UIRoot.SafeArea, false);
        tutGo.AddComponent&lt;TutorialHint&gt;();
    }
    if (_runner == null)
    {
        var sessionGo = new GameObject("[Game] Session");
        _runner = sessionGo.AddComponent&lt;LevelRunner&gt;();
        sessionGo.AddComponent&lt;GoalTracker&gt;();
        _timer = sessionGo.AddComponent&lt;LevelTimer&gt;();
    }
}
private static void OnStartRequested(LevelStartRequested e)
{
    // W4-G         0         +            ——
    //       OnDailyRequested         campaign
    LevelProgressStore.MaxLevelReached = e.LevelIndex;
    var all = ContentCatalog.All();
    if (e.LevelIndex &lt; 0 || e.LevelIndex &gt;= all.Length)
    {
        Debug.Log("[Game] LevelIndex " + e.LevelIndex + "                ");
        GameStateMachine.Set(GameState.Menu);
        return;
    }
    _runner.StartLevel(all[e.LevelIndex]);
}
private static void OnContinueRequested(ContinueRequested e)
{
    // "    "=     W4          UI
    //    MaxLevelReached=0 → L1
    var all = ContentCatalog.All();
    int idx = Mathf.Clamp(LevelProgressStore.MaxLevelReached, 0, all.Length - 1);
    _runner.StartLevel(all[idx]);
}
private static void OnDailyRequested(DailyLevelRequested e)
```

```
{
    var all = ContentCatalog.All();
    // W9             PlayableCount=30=         ch7-18
    int cap = Mathf.Min(all.Length, ContentCatalog.PlayableCount);
    int idx = DailyLevelPolicy.ResolveDaily(cap);
    if (idx &lt; 0 || idx &gt;= cap) return;
    _runner.StartLevel(all[idx]);
}
private static void OnReviveRequested(ReviveRequested e)
{
    if (_timer != null) _timer.Extend(ReviveSeconds);
    GameStateMachine.Set(GameState.Playing);   //      F       GameStateChanged
}
// W22-META           _timer     ——       MetaFlow
private static void OnItemUsedRequested(ItemUsedRequested e)
{
    // X1517 slice-f              MetaItem.Feather      ——
    if (e.Item != MetaItem.Hourglass &amp;&amp; e.Item != MetaItem.Magnet &amp;&amp; e.Item != MetaItem.Feather)
    if (!MetaStore.TryConsume(e.Item))
    {
        return;   //
    }
    // U314-P0-4               ——       1
    GameEventBus.Publish(new ItemEffectApplied { Item = e.Item });
    if (e.Item == MetaItem.Hourglass)
    {
        if (_timer != null) _timer.Extend(5f);   // +5s ReviveSeconds    Extend
    }
    else if (e.Item == MetaItem.Magnet)
    {
        SuctionField.BoostRadius(1.25f, 15f);   //      ×1.25    15s Gameplay     API
    }
    else if (e.Item == MetaItem.Feather)
    {
        // X1517 slice-f   =    ——             ReviveRequested
        // Bootstrap      Extend+  Playing   TelemetryRelay revive_used/revive_outcome
        //             ReviveRequested
        GameEventBus.Publish(new ReviveRequested());
    }
    GameEventBus.Publish(new ItemStockChanged { Item = e.Item, Count = MetaStore.ItemCount(e.Item) });
}
private static void OnReturnMenu(ReturnMenuRequested e)
{
    if (_runner != null) _runner.StopLevel();  //
}
private static void OnLevelStarted(LevelStarted e)
{
    //                    ChapterArt      Show(null)
    ApplyBackdrop(e.LevelId);
}
```

```
        // W12       DailyFocusResolved             UI     Core   GameEvents
        private static void OnGameStateChanged(GameStateChanged e)
        {
            if (e.Next == GameState.Menu) PublishDailyFocus();
        }
        private static void PublishDailyFocus()
        {
            var all = ContentCatalog.All();
            int cap = Mathf.Min(all.Length, ContentCatalog.PlayableCount);
            int idx = DailyLevelPolicy.ResolveDaily(cap);
            if (idx &lt; 0 || idx &gt;= cap) return;
            GameEventBus.Publish(new DailyFocusResolved
            {
                LevelId = all[idx].id,
                ChapterTitle = ChapterLore.GetChapterTitle(all[idx].id),
            });
        }
        // W10        scene → ShowLarge       CameraRig           Show(bg)
        // cover-fit                SceneBounds
        // [-9,9]×[-9,+1.5]                   →   =yMin+0.6h      6%
        // ≈1.24u         0.44×
        private static void ApplyBackdrop(int levelId)
        {
            if (_backdrop == null) return;
            Sprite scene = ChapterArt.Scene(levelId);
            if (scene != null) _backdrop.ShowLarge(scene);
            else _backdrop.Show(ChapterArt.For(levelId));
            Rect sb = _backdrop.SceneBounds;
            float inset = sb.width * 0.06f;
            float bandTop = sb.yMin + sb.height * 0.60f - inset * 0.44f;
            ActorDudu.WalkBounds = new Rect(
                sb.xMin + inset, sb.yMin + inset, sb.width - inset * 2f, bandTop - (sb.yMin + inset));
        }
    }
}
// GameplayArt.cs
using UnityEngine;
namespace GimmeAll.Gameplay
{
    /// &lt;summary&gt;
    /// Gameplay         W6-B1        Art/Resources     W10-E1   actor_dudu
    /// W11-NEWBIE-FUN       +    +
    ///              Resources.Load
    ///      getter =     +            GC
    ///       Debug.LogWarning       null      `?? PlaceholderSpriteFactory.X`
    ///   RuntimeInitializeOnLoad /            Menu
    /// &lt;/summary&gt;
    public static class GameplayArt
    {
        //             string         Count
```

```
private enum Slot
{
    Dust = 0, Fur, Debris, Spark, RingGlow, ActorDudu,
    SigSock, SigFurball, SigSnake, SigCoins, SigFuzzhill,   // W11
    Fuzz, DuduStunned, DuduEcstatic,                         // W11    +
    ScratchTea, ScratchCoffee,                              // U360-L7           · 8   / 13
    Count
}
// Resources   = Assets/_Project/Gameplay/Art/Resources/  =
private const string DustAsset = "target_dust";
private const string FurAsset = "target_fur";
private const string DebrisAsset = "target_debris";
private const string SparkAsset = "spark_soft";
private const string RingGlowAsset = "ring_glow";
private const string ActorDuduAsset = "actor_dudu";   // W10-E1
private const string SigSockAsset = "sig_sock";           // W11  1
private const string SigFurballAsset = "sig_furball";     // W11  2
private const string SigSnakeAsset = "sig_snake";         // W11  3
private const string SigCoinsAsset = "sig_coins";         // W11  4    +
private const string SigFuzzhillAsset = "sig_fuzzhill";    // W11  5
private const string FuzzAsset = "target_fuzz";            // W11      fuzz     target_*
private const string DuduStunnedAsset = "actor_dudu_stunned";   // W11 §   D
private const string DuduEcstaticAsset = "actor_dudu_ecstatic"; // W11 §   D
private const string ScratchTeaAsset = "scratch_tea";       // U360-L7     8   ·PIL
private const string ScratchCoffeeAsset = "scratch_coffee"; // U360-L7      13
private static readonly Sprite[] _cache = new Sprite[(int)Slot.Count];
private static bool _loaded;
public static Sprite Dust { get { EnsureLoaded(); return _cache[(int)Slot.Dust]; } }
public static Sprite Fur { get { EnsureLoaded(); return _cache[(int)Slot.Fur]; } }
public static Sprite Debris { get { EnsureLoaded(); return _cache[(int)Slot.Debris]; } }
public static Sprite Spark { get { EnsureLoaded(); return _cache[(int)Slot.Spark]; } }
public static Sprite RingGlow { get { EnsureLoaded(); return _cache[(int)Slot.RingGlow]; } }
public static Sprite ActorDudu { get { EnsureLoaded(); return _cache[(int)Slot.ActorDudu]; } }
// W11-NEWBIE-FUN    +  +      =null →      /     TestArtAssets
public static Sprite SigSock { get { EnsureLoaded(); return _cache[(int)Slot.SigSock]; } }
public static Sprite SigFurball { get { EnsureLoaded(); return _cache[(int)Slot.SigFurball]; } }
public static Sprite SigSnake { get { EnsureLoaded(); return _cache[(int)Slot.SigSnake]; } }
public static Sprite SigCoins { get { EnsureLoaded(); return _cache[(int)Slot.SigCoins]; } }
public static Sprite SigFuzzhill { get { EnsureLoaded(); return _cache[(int)Slot.SigFuzzhill]; } }
public static Sprite Fuzz { get { EnsureLoaded(); return _cache[(int)Slot.Fuzz]; } }
public static Sprite DuduStunned { get { EnsureLoaded(); return _cache[(int)Slot.DuduStunned]; } }
public static Sprite DuduEcstatic { get { EnsureLoaded(); return _cache[(int)Slot.DuduEcstatic]; } }
// U360-L7        =null →           ·TestArtAssets
public static Sprite ScratchTea { get { EnsureLoaded(); return _cache[(int)Slot.ScratchTea]; } }
public static Sprite ScratchCoffee { get { EnsureLoaded(); return _cache[(int)Slot.ScratchCoffee]; } }
private static void EnsureLoaded()
{
    if (_loaded) return;
    _loaded = true;   //                          null =
    _cache[(int)Slot.Dust] = Load(DustAsset);
```

```
            _cache[(int)Slot.Fur] = Load(FurAsset);
            _cache[(int)Slot.Debris] = Load(DebrisAsset);
            _cache[(int)Slot.Spark] = Load(SparkAsset);
            _cache[(int)Slot.RingGlow] = Load(RingGlowAsset);
            _cache[(int)Slot.ActorDudu] = Load(ActorDuduAsset);
            _cache[(int)Slot.SigSock] = Load(SigSockAsset);
            _cache[(int)Slot.SigFurball] = Load(SigFurballAsset);
            _cache[(int)Slot.SigSnake] = Load(SigSnakeAsset);
            _cache[(int)Slot.SigCoins] = Load(SigCoinsAsset);
            _cache[(int)Slot.SigFuzzhill] = Load(SigFuzzhillAsset);
            _cache[(int)Slot.Fuzz] = Load(FuzzAsset);
            _cache[(int)Slot.DuduStunned] = Load(DuduStunnedAsset);
            _cache[(int)Slot.DuduEcstatic] = Load(DuduEcstaticAsset);
            _cache[(int)Slot.ScratchTea] = Load(ScratchTeaAsset);
            _cache[(int)Slot.ScratchCoffee] = Load(ScratchCoffeeAsset);
        }
        private static Sprite Load(string assetName)
        {
            Sprite s = Resources.Load&lt;Sprite&gt;(assetName);
            if (s == null)
            {
                //                            null
                //
                Debug.LogWarning("[GameplayArt] Resources.Load&lt;Sprite&gt;(\"" + assetName + "\")              ");
            }
            return s;
        }
    }
}
// GameplayDirtPalette.cs
using UnityEngine;
namespace GimmeAll.Gameplay
{
    /// &lt;summary&gt;
    /// U325         orders O-20260930-1435·    §   18   × {dust,fur,debris}
    ///       =54  ——                tintIdx 0=      1-18=
    ///             /     color = Lerp(  ,   , 0.35)   /spriteKey
    ///            0.35       W22-META    tint       V    ·U309
    ///                         ·      dust=   55%
    /// fur=   30%            debris=   45%   +10%
    ///           Create    tintIdx      Update
    /// &lt;/summary&gt;
    public static class GameplayDirtPalette
    {
        /// &lt;summary&gt;    ContentCatalog.ChapterCount     =90/5  &lt;/summary&gt;
        public const int ChapterCount = 18;
        // 18           ·       1-18   = /  /  /  /  /  /   /  /
        //     /  /   /  /  /   /  /  /  /
        private static readonly Color[] ChapterBase =
        {
```

```
            new Color(0.93f, 0.85f, 0.72f),   // 1  ·
            new Color(0.96f, 0.78f, 0.83f),   // 2   ·
            new Color(0.95f, 0.75f, 0.72f),   // 3   ·
            new Color(0.95f, 0.85f, 0.60f),   // 4   ·
            new Color(0.92f, 0.62f, 0.58f),   // 5   ·
            new Color(0.66f, 0.72f, 0.95f),   // 6   ·
            new Color(0.62f, 0.82f, 0.85f),   // 7    ·
            new Color(0.72f, 0.85f, 0.95f),   // 8   ·
            new Color(0.80f, 0.90f, 0.96f),   // 9     ·
            new Color(0.60f, 0.78f, 0.68f),   // 10   ·
            new Color(0.55f, 0.87f, 0.85f),   // 11    ·
            new Color(0.72f, 0.62f, 0.92f),   // 12   ·
            new Color(0.95f, 0.72f, 0.45f),   // 13   ·
            new Color(0.78f, 0.68f, 0.92f),   // 14    ·
            new Color(0.94f, 0.62f, 0.45f),   // 15   ·
            new Color(0.92f, 0.78f, 0.72f),   // 16   ·
            new Color(0.68f, 0.85f, 0.95f),   // 17   ·
            new Color(0.96f, 0.85f, 0.55f),   // 18   ·
        };
        /// &lt;summary&gt; →     dust 55%   /fur 30%   /debris 45%   +   ——
        /// typeKey        /                    &lt;/summary&gt;
        public static Color TintOf(string typeKey, int chapterIdx)
        {
            if (chapterIdx &lt; 1 || chapterIdx &gt; ChapterCount) return Color.white;
            Color c = ChapterBase[chapterIdx - 1];
            if (typeKey == "dust") return new Color(c.r * 0.55f + 0.45f, c.g * 0.55f + 0.45f, c.b * 0.55f + 0.45f);
            if (typeKey == "fur") return new Color(c.r * 0.30f + 0.70f, c.g * 0.30f + 0.70f, c.b * 0.30f + 0.70f);
            if (typeKey == "debris")
            {
                //     10%                 45%
                float mean = (c.r + c.g + c.b) / 3f;
                float dr = c.r + (mean - c.r) * 0.1f;
                float dg = c.g + (mean - c.g) * 0.1f;
                float db = c.b + (mean - c.b) * 0.1f;
                return new Color(dr * 0.45f + 0.55f, dg * 0.45f + 0.55f, db * 0.45f + 0.55f);
            }
            return Color.white;
        }
    }
}
// GameplaySelfTest.cs
#if UNITY_EDITOR
using System.Collections.Generic;
using UnityEngine;
using GimmeAll.Core;
namespace GimmeAll.Gameplay
{
    /// &lt;summary&gt;
    /// A             Play           + 3      (45  ) +     +     +
    /// OnGUI       (width-310, 10, 300, 450) radius/strength/followLerp
```

```
///    W9-B     (count/Fill01/Ready) +   +1 /     (    SneezeRequested) /
///     W10-E1  FollowActor        =             +
/// &lt;/summary&gt;
public class GameplaySelfTest : MonoBehaviour
{
    private static readonly string[] CurveNames =
    {
        "  :    1-t",
        "  :    1-t²",
        "  :    (1-t)²"
    };
    private static bool _booted;
    private static int _capturedCount;
    private static int _escapedCount;
    private static readonly System.Random Rng = new System.Random(20260913);
    private static readonly List&lt;MonoBehaviour&gt; DestroyBuf = new List&lt;MonoBehaviour&gt;(256);
    private SuctionFieldConfig _config;
    private int _curveMode;
    private float _guiY;
    private SneezeSystem _sneeze;   //     W9-B  Play       /
    private SuctionField _field;     //     W10-E1  FollowActor
    private ActorDudu _actor;       //     W10-E1  Play     Ensure
    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
    private static void Boot()
    {
        // W6      + W18            Play          =
        //         vh_rigs_enabled    GimmeAll/  /
        if (UnityEditor.SessionState.GetBool("vh_playgate_active", false) ||
            !UnityEditor.SessionState.GetBool("vh_rigs_enabled", false)) return;
        if (_booted) return;   //
        _booted = true;
        EnsureCamera();
        GameObject go = new GameObject("GameplaySelfTest");
        go.AddComponent&lt;GameplaySelfTest&gt;();   // OnGUI    MonoBehaviour
    }
    private void Awake()
    {
        _config = ScriptableObject.CreateInstance&lt;SuctionFieldConfig&gt;();
        GameObject fieldGo = new GameObject("SuctionField");
        SuctionField field = fieldGo.AddComponent&lt;SuctionField&gt;();
        field.Config = _config;
        _field = field;
        GameEventBus.Subscribe&lt;ObjectCaptured&gt;(OnCapturedEvent);
        GameEventBus.Subscribe&lt;ObjectEscaped&gt;(OnEscapedEvent);
        _sneeze = SneezeSystem.Ensure();   //            Ensure
        _actor = ActorDudu.Ensure();       //      W10-E1              Ensure
        Regen();
    }
    private static void EnsureCamera()
    {
```

```
        if (Camera.main != null) return;
        GameObject go = new GameObject("SelfTestCamera");
        go.tag = "MainCamera";
        Camera cam = go.AddComponent&lt;Camera&gt;();
        cam.orthographic = true;
        cam.orthographicSize = 5f;
        cam.clearFlags = CameraClearFlags.SolidColor;
        cam.backgroundColor = new Color(0.07f, 0.08f, 0.11f, 1f);
        go.transform.position = new Vector3(0f, 0f, -10f);
    }
    private static void OnCapturedEvent(ObjectCaptured evt) { _capturedCount++; }
    private static void OnEscapedEvent(ObjectEscaped evt) { _escapedCount++; }
    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.R)) Regen();
    }
    private void Regen()
    {
        //                  Destroy
        DestroyBuf.Clear();
        int n = ObjectRegistry.Count;
        for (int i = 0; i &lt; n; i++)
        {
            if (ObjectRegistry.GetAt(i) is SuctionTarget t) DestroyBuf.Add(t);
        }
        for (int i = 0; i &lt; DestroyBuf.Count; i++) Destroy(DestroyBuf[i].gameObject);
        _capturedCount = 0;
        _escapedCount = 0;
        _sneeze.ResetCharge();   //     =
        // 3          9 dust + 4 fur + 2 debris = 45
        SpawnCluster(-4.2f, -2.2f);
        SpawnCluster(4.2f, -1.6f);
        SpawnCluster(0f, 2.4f);
    }
    private static void SpawnCluster(float cx, float cy)
    {
        for (int i = 0; i &lt; 9; i++) SpawnOne(GameplaySpawnFactory.DustKey, cx, cy, 1.7f, 0.32f, 0.5f);
        for (int i = 0; i &lt; 4; i++) SpawnOne(GameplaySpawnFactory.FurKey, cx, cy, 1.5f, 0.85f, 1.15f);
        for (int i = 0; i &lt; 2; i++) SpawnOne(GameplaySpawnFactory.DebrisKey, cx, cy, 1.3f, 0.9f, 1.1f);
    }
    private static void SpawnOne(string key, float cx, float cy, float spread, float scaleMin, float scaleMax)
    {
        SpawnEntry entry = new SpawnEntry();
        entry.typeKey = key;
        double ang = Rng.NextDouble() * System.Math.PI * 2.0;
        double rad = Rng.NextDouble() * spread;
        entry.x = cx + (float)(System.Math.Cos(ang) * rad);
        entry.y = cy + (float)(System.Math.Sin(ang) * rad);
        entry.scale = Mathf.Lerp(scaleMin, scaleMax, (float)Rng.NextDouble());
        SpawnRegistry.TryCreate(key, entry, out _);
```

```
}
private void OnGUI()
{
    GUI.Box(new Rect(Screen.width - 310f, 10f, 300f, 450f), "A      ");
    _guiY = 38f;
    GUI.Label(NextRow(1f), "      =    | R =     ");
    GUI.Label(NextRow(1f), "   " + _capturedCount + " |    " + _escapedCount + " |    " + ObjectRegistry.Count);
    _config.radius = GUI.HorizontalSlider(NextRow(1f), _config.radius, 0.5f, 6f);
    GUI.Label(NextRow(0.7f), "radius " + _config.radius.ToString("F2"));
    _config.baseStrength = GUI.HorizontalSlider(NextRow(1f), _config.baseStrength, 1f, 40f);
    GUI.Label(NextRow(0.7f), "baseStrength " + _config.baseStrength.ToString("F1"));
    _config.followLerp = GUI.HorizontalSlider(NextRow(1f), _config.followLerp, 0.05f, 1f);
    GUI.Label(NextRow(0.7f), "followLerp " + _config.followLerp.ToString("F2"));
    if (GUI.Button(NextRow(1f), CurveNames[_curveMode])) CycleCurve();
    if (GUI.Button(NextRow(1f), "     (45   )")) Regen();
    // ——     W10-E1  FollowActor    +         ——
    if (GUI.Button(NextRow(1f), _field.FollowActor
        ? "  :      =   (    )"
        : "  :      =   (   )"))
    {
        _field.FollowActor = !_field.FollowActor;
    }
    float toTarget = _actor != null ? Vector2.Distance(_actor.Pos, _actor.TargetPos) : -1f;
    GUI.Label(NextRow(1f), "       " + toTarget.ToString("F2")
        + (toTarget &gt;= 0f &amp;&amp; toTarget &lt; 0.05f ? " |    " : " |    "));
    GUI.Label(NextRow(1f), "  " + _sneeze.ChargeCount + "/" + _sneeze.Needed
        + " | Fill " + _sneeze.Fill01.ToString("F2")
        + (_sneeze.Ready ? " | READY" : " |    "));
    if (GUI.Button(NextRow(1f), "  +1   ")) _sneeze.AddCharge();
    if (GUI.Button(NextRow(1f), "    ")) GameEventBus.Publish(new SneezeRequested());
    if (GUI.Button(NextRow(1f), "    ")) _sneeze.ResetCharge();
}
private Rect NextRow(float lines)
{
    float h = 22f * lines;
    Rect r = new Rect(Screen.width - 300f, _guiY, 280f, h);
    _guiY += h + 3f;
    return r;
}
private void CycleCurve()
{
    _curveMode = (_curveMode + 1) % 3;
    switch (_curveMode)
    {
        case 0: _config.falloffCurve = AnimationCurve.Linear(0f, 1f, 1f, 0f); break;
        // 1-t² t=0     0 t=1     -2            Keyframe
        case 1: _config.falloffCurve = new AnimationCurve(
            new Keyframe(0f, 1f, 0f, 0f),
            new Keyframe(1f, 0f, -2f, -2f)); break;
        // (1-t)² t=0     -2 t=1     0
```

```
                default: _config.falloffCurve = new AnimationCurve(
                    new Keyframe(0f, 1f, -2f, -2f),
                    new Keyframe(1f, 0f, 0f, 0f)); break;
            }
        }
    }
}
#endif
// GameplaySpawnFactory.cs
using System.Collections.Generic;
using UnityEngine;
using GimmeAll.Core;
namespace GimmeAll.Gameplay
{
    /// &lt;summary&gt;
    /// "dust"/"fur"/"debris" + W11        ISpawnFactory B      SpawnRegistry
    /// W6-B1      GameplayArt         color=white    tint
    /// W11-NEWBIE-FUN sock/furball/snake/coins/fuzzhill      §       + fuzz
    ///           FuzzHillTarget             FuzzFade
    /// U325      v2 O-20260930-1435       + splitter      splitkid
    ///  bubble/splitter/splitkid/mimic/sparkle/heavy/ghost/swarm/sticky      tintIdx
    ///  Lerp 0.35  ·       /spriteKey        +     gearKey  Gear
    /// props            : [0]=mass, [1]=captureThreshold, [2]=maxSpeed
    /// BeforeSceneLoad            /
    /// &lt;/summary&gt;
    public class GameplaySpawnFactory : ISpawnFactory
    {
        // const=      switch case      CS0150  public=TestSignatureLevels
        public const string DustKey = "dust";
        public const string FurKey = "fur";
        public const string DebrisKey = "debris";
        // W11     §  S1  / 1-5
        public const string SockKey = "sock";
        public const string FurBallKey = "furball";
        public const string SnakeKey = "snake";
        public const string CoinsKey = "coins";
        public const string FuzzHillKey = "fuzzhill";
        public const string FuzzKey = "fuzz";
        // U325         §  + splitter
        public const string BubbleKey = "bubble";
        public const string SplitterKey = "splitter";
        public const string SplitKidKey = "splitkid";
        public const string MimicKey = "mimic";
        public const string SparkleKey = "sparkle";
        public const string HeavyKey = "heavy";
        public const string GhostKey = "ghost";
        public const string SwarmKey = "swarm";
        public const string StickyKey = "sticky";
        // U360-L3/L6     O-20261008-1438          L6 § .1 +       L3 §   3
        public const string ScratchTeaKey = "scratch_tea";
```

```
public const string ScratchCoffeeKey = "scratch_coffee";
public const string LandmarkKey = "landmark";
private readonly string _typeKey;
public string TypeKey { get { return _typeKey; } }
public GameplaySpawnFactory(string typeKey)
{
    _typeKey = typeKey;
}
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
private static void RegisterAll()
{
    SpawnRegistry.Register(new GameplaySpawnFactory(DustKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(FurKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(DebrisKey));
    // W11-NEWBIE-FUN      +      LevelRunner.CountRegistered       TotalTargets
    //    =
    SpawnRegistry.Register(new GameplaySpawnFactory(SockKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(FurBallKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(SnakeKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(CoinsKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(FuzzHillKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(FuzzKey));
    // U325      + splitkid     splitkid        ——
    // SplitterTarget    TotalTargets   ContentCatalog.CountWeightOf
    SpawnRegistry.Register(new GameplaySpawnFactory(BubbleKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(SplitterKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(SplitKidKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(MimicKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(SparkleKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(HeavyKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(GhostKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(SwarmKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(StickyKey));
    // U360-L3/L6     +    ContentCatalog.ApplyBeatInjections    ——   =
    //
    SpawnRegistry.Register(new GameplaySpawnFactory(ScratchTeaKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(ScratchCoffeeKey));
    SpawnRegistry.Register(new GameplaySpawnFactory(LandmarkKey));
}
public ISuctionable Create(in SpawnEntry entry)
{
    GameObject go = new GameObject(_typeKey);   //   =
    SpriteRenderer sr = go.AddComponent&lt;SpriteRenderer&gt;();   //            Awake
    // W6-B1          GameplayArt                    GC
    //                 tint    →      color=white
    // W11            /  =              =     GameplayArt      null
    // U325             2    →        mimic   =      dust
    Sprite art;
    Color fallbackColor;
    float baseSize;
```

```
switch (_typeKey)
{
    case SockKey:
        art = GameplayArt.SigSock; fallbackColor = new Color(0.65f, 0.92f, 0.80f, 1f); baseSize = 0.6f; brea
    case FurBallKey:
        art = GameplayArt.SigFurball; fallbackColor = new Color(1f, 0.55f, 0.2f, 1f); baseSize = 1.1f; break
    case SnakeKey:
        art = GameplayArt.SigSnake; fallbackColor = new Color(0.5f, 0.48f, 0.52f, 1f); baseSize = 1.0f; brea
    case CoinsKey:
        art = GameplayArt.SigCoins; fallbackColor = new Color(1f, 0.84f, 0.38f, 1f); baseSize = 0.8f; break;
    case FuzzHillKey:
        art = GameplayArt.SigFuzzhill; fallbackColor = new Color(1f, 0.95f, 0.85f, 1f); baseSize = 1.2f; bre
    case FuzzKey:
        //             fur
        art = GameplayArt.Fuzz ?? GameplayArt.Fur;
        fallbackColor = new Color(1f, 0.95f, 0.85f, 1f); baseSize = 0.35f; break;
    case DebrisKey:
        art = GameplayArt.Debris; fallbackColor = new Color(0.28f, 0.3f, 0.36f, 1f); baseSize = 0.9f; break;
    case FurKey:
        art = GameplayArt.Fur; fallbackColor = new Color(1f, 0.55f, 0.2f, 1f); baseSize = 0.7f; break;
    // ── U325           =     mimic       spriteKey         ──
    case BubbleKey:
        art = null; fallbackColor = new Color(0.72f, 0.9f, 1f, 1f); baseSize = 0.5f; break;
    case SplitterKey:
        art = null; fallbackColor = new Color(1f, 0.62f, 0.72f, 1f); baseSize = 0.8f; break;
    case SplitKidKey:
        art = null; fallbackColor = new Color(1f, 0.62f, 0.72f, 1f); baseSize = 0.4f; break;
    case MimicKey:
        //    =        Setup         OnSuctionStarted   ──    §
        art = GameplayArt.Dust; fallbackColor = new Color(0.82f, 0.82f, 0.86f, 1f); baseSize = 0.5f; break;
    case SparkleKey:
        art = null; fallbackColor = new Color(1f, 0.88f, 0.45f, 1f); baseSize = 0.45f; break;
    case HeavyKey:
        art = null; fallbackColor = new Color(0.5f, 0.55f, 0.66f, 1f); baseSize = 1.0f; break;
    case GhostKey:
        art = null; fallbackColor = new Color(0.85f, 0.82f, 0.98f, 1f); baseSize = 0.6f; break;
    case SwarmKey:
        art = null; fallbackColor = new Color(1f, 0.76f, 0.3f, 1f); baseSize = 0.4f; break;
    case StickyKey:
        art = null; fallbackColor = new Color(0.96f, 0.86f, 0.9f, 1f); baseSize = 0.65f; break;
    // ── U360-L3/L6          /      +           ──
    //    =                =       ──
    case ScratchTeaKey:
        art = GameplayArt.ScratchTea; fallbackColor = new Color(0.62f, 0.42f, 0.24f, 1f); baseSize = 0.78f;
    case ScratchCoffeeKey:
        art = GameplayArt.ScratchCoffee; fallbackColor = new Color(0.38f, 0.26f, 0.18f, 1f); baseSize = 0.78
    case LandmarkKey:
        art = null; fallbackColor = new Color(0.95f, 0.82f, 0.55f, 1f); baseSize = 1.1f; break;
    default:
        art = GameplayArt.Dust; fallbackColor = new Color(0.82f, 0.82f, 0.86f, 1f); baseSize = 0.45f; break;
```

```
        PlayerPrefs.SetInt(SneezeKey, 1);
        PlayerPrefs.Save();
    }
    private void Awake()
    {
        //            raycastTarget CreatePanel    true=
        var panel = UIRoot.CreatePanel(transform, new Color(0f, 0f, 0f, 0f));
        panel.raycastTarget = false;
        _root = panel.rectTransform;
        _bigText = UIRoot.CreateText(_root, "    ", 72, UIRoot.Ink);
        _bigText.font = UIRoot.DisplayFont;   // X1436/B7: coaching hero line = display-font track (U278)
        UIRoot.AnchorCenter(_bigText.rectTransform, new Vector2(0f, -240f), new Vector2(900f, 140f));
        AddLegibilityShadow(_bigText);   // U360-L7      BEFORE   =           ·
        _subText = UIRoot.CreateText(_root, "           ", 40, UIRoot.CoralDeep);   // U320    Coral       2.14
        UIRoot.AnchorCenter(_subText.rectTransform, new Vector2(0f, -360f), new Vector2(900f, 70f));
        AddLegibilityShadow(_subText);   // U360-L7      BEFORE     =      ·
        //         128²     CollectionScreen        ——
        var rippleGo = new GameObject("Ripple", typeof(RectTransform));
        _ripple = (RectTransform)rippleGo.transform;
        _ripple.SetParent(_bigText.rectTransform, false);
        UIRoot.AnchorCenter(_ripple, new Vector2(0f, 110f), new Vector2(150f, 150f));
        var img = rippleGo.AddComponent&lt;Image&gt;();
        img.sprite = BuildRippleSprite();
        img.raycastTarget = false;   //
        img.color = new Color(1f, 0.52f, 0.38f, 0.88f);   // U360-L7       AFTER        0.95        =           =
        // X2344 r2 tweak 4: outer diffusion ring — child of the ripple so it follows the level-1
        // re-parent (PositionRippleForLevel) and reads as one feedback family; alpha driven by the pulse loop.
        var ringGo = new GameObject("RippleRing", typeof(RectTransform));
        _rippleRing = (RectTransform)ringGo.transform;
        _rippleRing.SetParent(_ripple, false);
        UIRoot.AnchorCenter(_rippleRing, Vector2.zero, new Vector2(150f, 150f));
        _rippleRingImg = ringGo.AddComponent&lt;Image&gt;();
        _rippleRingImg.sprite = BuildRingSprite();
        _rippleRingImg.raycastTarget = false;   // redline: ring must not eat presses
        _rippleRingImg.color = new Color(1f, 0.52f, 0.38f, 0f);
        // U360-L7     toast        _root——       03   toast
        // 04             _root SetActive(false)            toast
        //      =     →_root  →     toast             =
        _toast = UIRoot.CreateText(transform, "    ——    ", 38, UIRoot.CoralDeep);   // U320           Coral 2.14→
        UIRoot.AnchorBottom(_toast.rectTransform, 330f, new Vector2(520f, 50f), 230f);
        _toast.gameObject.SetActive(false);   //         _root   ——       W21-UX
        _root.gameObject.SetActive(false);
    }
    private void OnEnable()
    {
        GameEventBus.Subscribe&lt;LevelStarted&gt;(OnLevelStarted);
        GameEventBus.Subscribe&lt;LevelFailed&gt;(OnLevelFailed);
        GameEventBus.Subscribe&lt;ObjectCaptured&gt;(OnObjectCaptured);
        GameEventBus.Subscribe&lt;SneezeChargeChanged&gt;(OnSneezeCharge);
    }
```

```
private void OnDisable()
{
    GameEventBus.Unsubscribe&lt;LevelStarted&gt;(OnLevelStarted);
    GameEventBus.Unsubscribe&lt;LevelFailed&gt;(OnLevelFailed);
    GameEventBus.Unsubscribe&lt;ObjectCaptured&gt;(OnObjectCaptured);
    GameEventBus.Unsubscribe&lt;SneezeChargeChanged&gt;(OnSneezeCharge);
    StopHintRoutine();
}
private void Update()
{
    if (!_hintActive) return;
    //        legacy Input   /        legacy Input WASM
    bool held = Input.GetMouseButton(0) || Input.touchCount &gt; 0;
    _holdTimer = held ? _holdTimer + Time.deltaTime : 0f;
    if (_holdTimer &gt;= HoldSeconds)
    {
        HideHint();
        MarkPressSeen();
    }
}
private void OnLevelStarted(LevelStarted e)
{
    StopHintRoutine();
    _root.gameObject.SetActive(false);
    _hintActive = false;
    if (PressSeen()) return;
    _hintLevelId = e.LevelId;   // U323 T3   1
    _hintRoutine = StartCoroutine(ShowHintAfterDelay());
}
private void OnLevelFailed(LevelFailed e)
{
    //              =
    StopHintRoutine();
    _root.gameObject.SetActive(false);
    _hintActive = false;
}
private void OnObjectCaptured(ObjectCaptured e)
{
    //     =
    if (!_hintActive) return;
    HideHint();
    MarkPressSeen();
}
private void OnSneezeCharge(SneezeChargeChanged e)
{
    if (!e.Ready || SneezeSeen()) return;
    MarkSneezeSeen();
    if (_toastActive) return;
    StartCoroutine(ToastRoutine());
}
```

```
private IEnumerator ShowHintAfterDelay()
{
    yield return new WaitForSeconds(StartDelaySeconds);   //
    _root.gameObject.SetActive(true);
    _hintActive = true;
    _holdTimer = 0f;
    PositionRippleForLevel();   // U323 T3        1=   B      ·   =
    //    +                GC
    float t = 0f;
    while (_hintActive &amp;&amp; t &lt; 0.35f)
    {
        t += Time.deltaTime;
        SetHintAlpha(Mathf.Clamp01(t / 0.35f));
        yield return null;
    }
    while (_hintActive)
    {
        float pulse = 0.82f + 0.18f * Mathf.Sin(Time.unscaledTime * 3.2f);
        _ripple.localScale = new Vector3(pulse, pulse, 1f);
        // X2344 r2 tweak 4: continuous outward diffusion — ring scales 1.0 -&gt; 1.55 while alpha fades 0.40 -&
        float ringPhase = (Time.unscaledTime * 0.85f) % 1f;
        float ringScale = 1f + 0.55f * ringPhase;
        _rippleRing.localScale = new Vector3(ringScale, ringScale, 1f);
        _rippleRingImg.color = new Color(1f, 0.52f, 0.38f, 0.40f * (1f - ringPhase));
        yield return null;
    }
}
/// &lt;summary&gt;U323 T3           1               B     (1.5,-2.9)
///     ——               R   G3   =   →               &lt;/summary&gt;
private void PositionRippleForLevel()
{
    if (_hintLevelId == 1 &amp;&amp; Camera.main != null)
    {
        Vector2 canvasPos = FlyToTarget.WorldToCanvas(new Vector3(1.5f, -2.9f, 0f));
        _ripple.SetParent(_root, false);
        UIRoot.AnchorCenter(_ripple, canvasPos, new Vector2(150f, 150f));
        _ripple.SetAsFirstSibling();   // X2333  1              ——B                            AFTER
    }
    else
    {
        _ripple.SetParent(_bigText.rectTransform, false);
        UIRoot.AnchorCenter(_ripple, new Vector2(0f, 110f), new Vector2(150f, 150f));
    }
}
private void HideHint()
{
    _hintActive = false;
    StopHintRoutine();
    StartCoroutine(FadeOutAndHide());
}
```

```
private IEnumerator FadeOutAndHide()
{
    float t = 0f;
    float a = CurrentAlpha();
    while (t &lt; 0.25f)
    {
        t += Time.deltaTime;
        SetHintAlpha(a * (1f - Mathf.Clamp01(t / 0.25f)));
        yield return null;
    }
    _root.gameObject.SetActive(false);
    SetHintAlpha(1f);
}
private IEnumerator ToastRoutine()
{
    _toastActive = true;
    _toast.gameObject.SetActive(true);
    float t = 0f;
    while (t &lt; ToastSeconds)
    {
        t += Time.deltaTime;
        float k = Mathf.Clamp01(t / ToastSeconds);
        float a = k &lt; 0.15f ? k / 0.15f : (k &gt; 0.85f ? (1f - k) / 0.15f : 1f);
        SetAlpha(_toast, a);
        yield return null;
    }
    _toast.gameObject.SetActive(false);
    SetAlpha(_toast, 1f);
    _toastActive = false;
}
private void StopHintRoutine()
{
    if (_hintRoutine == null) return;
    StopCoroutine(_hintRoutine);
    _hintRoutine = null;
}
/// &lt;summary&gt;U360-L7        W24-PLAY-SEE   ——       /          &lt;/summary&gt;
private static void AddLegibilityShadow(Text t)
{
    var sh = t.gameObject.AddComponent&lt;Shadow&gt;();
    sh.effectColor = new Color(0.345f, 0.245f, 0.200f, 0.45f);
    sh.effectDistance = new Vector2(2f, -2f);
}
private float CurrentAlpha()
{
    return _bigText.color.a;
}
private void SetHintAlpha(float a)
{
    SetAlpha(_bigText, a);
```

```
    SetAlpha(_subText, a);
    SetAlpha(_ripple.GetComponent&lt;Image&gt;(), a);
}
private static void SetAlpha(Graphic g, float a)
{
    var c = g.color;
    c.a = a;
    g.color = c;
}
/// &lt;summary&gt;    128²             Image.color  ——CollectionScreen       &lt;/summary&gt;
private static Sprite BuildRippleSprite()
{
    const int Size = 128;
    const float Radius = 56f;
    const float Edge = 6f;
    var tex = new Texture2D(Size, Size, TextureFormat.RGBA32, false);
    tex.wrapMode = TextureWrapMode.Clamp;
    tex.filterMode = FilterMode.Bilinear;
    var pixels = new Color32[Size * Size];
    int k = 0;
    for (int y = 0; y &lt; Size; y++)
    {
        for (int x = 0; x &lt; Size; x++)
        {
            float dx = x + 0.5f - Size * 0.5f;
            float dy = y + 0.5f - Size * 0.5f;
            float dist = Mathf.Sqrt(dx * dx + dy * dy);
            float alpha = Mathf.Clamp01((Radius - dist) / Edge);
            pixels[k++] = new Color32(255, 255, 255, (byte)(alpha * 255f + 0.5f));
        }
    }
    tex.SetPixels32(pixels);
    tex.Apply(false, false);
    var sprite = Sprite.Create(tex, new Rect(0f, 0f, Size, Size),
        new Vector2(Size * 0.5f, Size * 0.5f), 100f);
    sprite.name = "tut_ripple_circle";
    return sprite;
}
/// &lt;summary&gt;X2344 r2 tweak 4: thin soft ring sprite for the outer diffusion halo (self-sufficient, zero a
private static Sprite BuildRingSprite()
{
    const int Size = 128;
    const float Outer = 60f;
    const float Inner = 47f;
    const float Edge = 5f;
    var tex = new Texture2D(Size, Size, TextureFormat.RGBA32, false);
    tex.wrapMode = TextureWrapMode.Clamp;
    tex.filterMode = FilterMode.Bilinear;
    var pixels = new Color32[Size * Size];
    int k = 0;
```

```
            for (int y = 0; y &lt; Size; y++)
            {
                for (int x = 0; x &lt; Size; x++)
                {
                    float dx = x + 0.5f - Size * 0.5f;
                    float dy = y + 0.5f - Size * 0.5f;
                    float dist = Mathf.Sqrt(dx * dx + dy * dy);
                    float outerFade = Mathf.Clamp01((Outer - dist) / Edge);
                    float innerFade = Mathf.Clamp01((dist - Inner) / Edge);
                    float alpha = Mathf.Min(outerFade, innerFade);
                    pixels[k++] = new Color32(255, 255, 255, (byte)(alpha * 255f + 0.5f));
                }
            }
            tex.SetPixels32(pixels);
            tex.Apply(false, false);
            var sprite = Sprite.Create(tex, new Rect(0f, 0f, Size, Size),
                new Vector2(Size * 0.5f, Size * 0.5f), 100f);
            sprite.name = "tut_ripple_ring";
            return sprite;
        }
    }
}
// UIArt.cs
using System.Collections.Generic;
using UnityEngine;
namespace GimmeAll.UI
{
    /// &lt;summary&gt;
    /// W7-UI      Resources       ui_btn_pad / ui_panel_round          border
    ///               Sprite/  /PPU   border    Image.type=Sliced
    /// Sprite.Create          /         null——      W2-F
    ///                              BeforeSceneLoad
    /// W13-UI    Icon(key)      §   B        +     Resources.Load
    ///                =   null+             /
    /// &lt;/summary&gt;
    public static class UIArt
    {
        private const string ButtonPadName = "ui_btn_pad";
        private const string PanelRoundName = "ui_panel_round";
        private const string BannerLandingName = "ui_banner_landing";
        private const string PostcardFrameName = "ui_postcard_frame";
        private const string TagPillName = "ui_tag_pill";
        private const string LogoEmblemName = "ui_logo_emblem";
        private const string LogoMarkName = "logo_mark";
        private const string LogoWordmarkName = "logo_wordmark";
        private const string LogoHorizontalName = "logo_horizontal";
        private const string LogoVerticalName = "logo_vertical";
        private const string DuduFullName = "dudu_full";
        private const string HealthNoticeBgName = "bg_health_notice";
        private const string MainMenuBgName = "bg_main_menu";
```

```
// O-1610: watercolor paper bases (CEO verdict: flat-tint buttons clash with the painted
// background). Generated per U284 style-master SOP, cropped to content bbox on import.
private const string WcBtnPrimaryName = "wc_btn_primary";
private const string WcBtnSecondaryName = "wc_btn_secondary";
// O-1610 slice-b: capsule + panel watercolor bases. Their 9-slice borders are baked
// import-time (O1610BorderBake -&gt; spriteBorder meta, capsule-correct; the runtime 15%
// ratio exceeds capsule heights 64/76/88 — U322 verdict). LoadSliced honors the
// imported border when present; 15% stays the fallback for borderless legacy assets.
private const string WcPillName = "wc_pill";
private const string WcPanelName = "wc_panel";
//            12–18% W7           15%
//      70%                  ArtImportRules maxSize=512 →
//   1024²       512² 15% ≈ 77px
private const float BorderRatio = 0.15f;
private static Sprite _buttonPad;
private static Sprite _panelRound;
private static Sprite _bannerLanding;
private static Sprite _postcardFrame;
private static Sprite _tagPill;
private static Sprite _logoEmblem;
private static Sprite _logoMark;
private static Sprite _logoWordmark;
private static Sprite _logoHorizontal;
private static Sprite _logoVertical;
private static Sprite _duduFull;
private static Sprite _healthNoticeBg;
private static Sprite _mainMenuBg;
private static Sprite _wcBtnPrimary;
private static Sprite _wcBtnSecondary;
private static Sprite _wcPill;
private static Sprite _wcPanel;
private static bool _buttonPadTried;
private static bool _panelRoundTried;
private static bool _bannerLandingTried;
private static bool _postcardFrameTried;
private static bool _tagPillTried;
private static bool _logoEmblemTried;
private static bool _logoMarkTried;
private static bool _logoWordmarkTried;
private static bool _logoHorizontalTried;
private static bool _logoVerticalTried;
private static bool _duduFullTried;
private static bool _healthNoticeBgTried;
private static bool _mainMenuBgTried;
private static bool _wcBtnPrimaryTried;
private static bool _wcBtnSecondaryTried;
private static bool _wcPillTried;
private static bool _wcPanelTried;
// W13-UI      key   "daily" → "icon_daily"
private static readonly Dictionary&lt;string, Sprite&gt; _iconCache = new Dictionary&lt;string, Sprite&gt;(16);
```

```
/// &lt;summary&gt;      Sliced            null         &lt;/summary&gt;
public static Sprite ButtonPad
{
    get
    {
        if (!_buttonPadTried)
        {
            _buttonPadTried = true;   //
            _buttonPad = LoadSliced(ButtonPadName);
        }
        return _buttonPad;
    }
}
/// &lt;summary&gt;      Sliced            null         &lt;/summary&gt;
public static Sprite PanelRound
{
    get
    {
        if (!_panelRoundTried)
        {
            _panelRoundTried = true;
            _panelRound = LoadSliced(PanelRoundName);
        }
        return _panelRound;
    }
}
/// &lt;summary&gt;O-1610       CTA    Sliced   ·   wash         tint
///        null       ButtonPad+   tint      &lt;/summary&gt;
public static Sprite WcBtnPrimary
{
    get
    {
        if (!_wcBtnPrimaryTried)
        {
            _wcBtnPrimaryTried = true;
            _wcBtnPrimary = LoadSliced(WcBtnPrimaryName);
        }
        return _wcBtnPrimary;
    }
}
/// &lt;summary&gt;O-1610            Sliced   ·   wash         null &lt;/summary&gt;
public static Sprite WcBtnSecondary
{
    get
    {
        if (!_wcBtnSecondaryTried)
        {
            _wcBtnSecondaryTried = true;
            _wcBtnSecondary = LoadSliced(WcBtnSecondaryName);
        }
```

```
        return _wcBtnSecondary;
    }
}
/// &lt;summary&gt;O-1610 slice-b        Sliced   ·   wash·border=
///               null       ButtonPad+   tint       &lt;/summary&gt;
public static Sprite WcPill
{
    get
    {
        if (!_wcPillTried)
        {
            _wcPillTried = true;
            _wcPill = LoadSliced(WcPillName);
        }
        return _wcPill;
    }
}
/// &lt;summary&gt;O-1610 slice-b        Sliced   ·   wash·  /
///         ——                 null &lt;/summary&gt;
public static Sprite WcPanel
{
    get
    {
        if (!_wcPanelTried)
        {
            _wcPanelTried = true;
            _wcPanel = LoadSliced(WcPanelName);
        }
        return _wcPanel;
    }
}
/// &lt;summary&gt;O-1610 slice-e modal/overlay panel: watercolor wc_panel first (same paper-wash
/// family as WcBtn/WcPill -&gt; one visual lineage across buttons, capsules and modals),
/// R6 ui_panel_round as legacy fallback, then the caller's plain-color path. Tinted
/// chrome (halos, card bases, list beds) keeps PanelRound for clean dyeing.&lt;/summary&gt;
public static Sprite ModalPanel
{
    get { return WcPanel ?? PanelRound; }
}
/// &lt;summary&gt;            Sprite   Sliced         null &lt;/summary&gt;
public static Sprite BannerLanding
{
    get
    {
        if (!_bannerLandingTried)
        {
            _bannerLandingTried = true;
            _bannerLanding = LoadPlain(BannerLandingName);
        }
        return _bannerLanding;
```

```
    }
}
/// &lt;summary&gt;         Sprite     +             null &lt;/summary&gt;
public static Sprite PostcardFrame
{
    get
    {
        if (!_postcardFrameTried)
        {
            _postcardFrameTried = true;
            _postcardFrame = LoadPlain(PostcardFrameName);
        }
        return _postcardFrame;
    }
}
/// &lt;summary&gt;      Sliced         /            null &lt;/summary&gt;
public static Sprite TagPill
{
    get
    {
        if (!_tagPillTried)
        {
            _tagPillTried = true;
            _tagPill = LoadSliced(TagPillName);
        }
        return _tagPill;
    }
}
/// &lt;summary&gt;         Sprite               null &lt;/summary&gt;
public static Sprite LogoEmblem
{
    get
    {
        if (!_logoEmblemTried)
        {
            _logoEmblemTried = true;
            _logoEmblem = LoadPlain(LogoEmblemName);
        }
        return _logoEmblem;
    }
}
/// &lt;summary&gt;               +  +              null &lt;/summary&gt;
public static Sprite HealthNoticeBg
{
    get
    {
        if (!_healthNoticeBgTried)
        {
            _healthNoticeBgTried = true;
            _healthNoticeBg = LoadPlain(HealthNoticeBgName);
```

```
        }
        return _healthNoticeBg;
    }
}
/// &lt;summary&gt;           +    +             null &lt;/summary&gt;
public static Sprite MainMenuBg
{
    get
    {
        if (!_mainMenuBgTried)
        {
            _mainMenuBgTried = true;
            _mainMenuBg = LoadPlain(MainMenuBgName);
        }
        return _mainMenuBg;
    }
}
/// &lt;summary&gt;   Logo           +                   null &lt;/summary&gt;
public static Sprite LogoMark
{
    get
    {
        if (!_logoMarkTried) { _logoMarkTried = true; _logoMark = LoadPlain(LogoMarkName); }
        return _logoMark;
    }
}
/// &lt;summary&gt;     "Gimme All"      i =  +             null &lt;/summary&gt;
public static Sprite LogoWordmark
{
    get
    {
        if (!_logoWordmarkTried) { _logoWordmarkTried = true; _logoWordmark = LoadPlain(LogoWordmarkName); }
        return _logoWordmark;
    }
}
/// &lt;summary&gt;   Logo         +        /            null &lt;/summary&gt;
public static Sprite LogoHorizontal
{
    get
    {
        if (!_logoHorizontalTried) { _logoHorizontalTried = true; _logoHorizontal = LoadPlain(LogoHorizontalName
        return _logoHorizontal;
    }
}
/// &lt;summary&gt;   Logo         +         /             null &lt;/summary&gt;
public static Sprite LogoVertical
{
    get
    {
        if (!_logoVerticalTried) { _logoVerticalTried = true; _logoVertical = LoadPlain(LogoVerticalName); }
```

```
        return _logoVertical;
    }
}
/// &lt;summary&gt;             +   +                    null &lt;/summary&gt;
public static Sprite DuduFull
{
    get
    {
        if (!_duduFullTried) { _duduFullTried = true; _duduFull = LoadPlain(DuduFullName); }
        return _duduFull;
    }
}
/// &lt;summary&gt;
/// W13-UI      §   B    daily/rank/share/star/sneeze/codex/home/back/retry/revive
///       icon_prop_*                        null   ——
///     /                      /
/// &lt;/summary&gt;
public static Sprite Icon(string key)
{
    if (_iconCache.TryGetValue(key, out Sprite cached)) return cached;
    var sprite = Resources.Load&lt;Sprite&gt;("icon_" + key);
    if (sprite == null) Debug.LogWarning("[UIArt]               icon_" + key);
    _iconCache[key] = sprite;
    return sprite;
}
/// &lt;summary&gt;X1497             scene_*_thumb    Resources
/// Sprite      null         ——  7+             &lt;/summary&gt;
public static Sprite Plain(string name)
{
    return LoadPlain(name);
}
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.SubsystemRegistration)]
private static void ResetStatics()
{
    // G11 Fast-Enter
    //           null     Destroy
    if (_buttonPad != null) Object.Destroy(_buttonPad);
    if (_panelRound != null) Object.Destroy(_panelRound);
    if (_tagPill != null) Object.Destroy(_tagPill);
    if (_wcBtnPrimary != null) Object.Destroy(_wcBtnPrimary);
    if (_wcBtnSecondary != null) Object.Destroy(_wcBtnSecondary);
    if (_wcPill != null) Object.Destroy(_wcPill);
    if (_wcPanel != null) Object.Destroy(_wcPanel);
    _buttonPad = null;
    _panelRound = null;
    _bannerLanding = null;
    _postcardFrame = null;
    _tagPill = null;
    _logoEmblem = null;
    _logoMark = null;
```

```
        _logoWordmark = null;
        _logoHorizontal = null;
        _logoVertical = null;
        _duduFull = null;
        _healthNoticeBg = null;
        _mainMenuBg = null;
        _wcBtnPrimary = null;
        _wcBtnSecondary = null;
        _wcPill = null;
        _wcPanel = null;
        _buttonPadTried = false;
        _panelRoundTried = false;
        _bannerLandingTried = false;
        _postcardFrameTried = false;
        _tagPillTried = false;
        _logoEmblemTried = false;
        _logoMarkTried = false;
        _logoWordmarkTried = false;
        _logoHorizontalTried = false;
        _logoVerticalTried = false;
        _duduFullTried = false;
        _healthNoticeBgTried = false;
        _mainMenuBgTried = false;
        _wcBtnPrimaryTried = false;
        _wcBtnSecondaryTried = false;
        _wcPillTried = false;
        _wcPanelTried = false;
        //     =      Resources.Load      Sprite.Create
        _iconCache.Clear();
    }
    private static Sprite LoadSliced(string name)
    {
        //    Sprite ArtImportRules     Art/Resources        Sprite
        //     Texture2D    PNG            Default
        // O-1610 slice-b Sprite         border O1610BorderBake     spriteBorder
        //     ——border                 border     15%
        var src = Resources.Load&lt;Sprite&gt;(name);
        if (src != null) return CreateVariant(src.texture, src.textureRect, name, src.border);
        var tex = Resources.Load&lt;Texture2D&gt;(name);
        if (tex != null) return CreateVariant(tex, new Rect(0f, 0f, tex.width, tex.height), name, Vector4.zero);
        Debug.LogWarning("[UIArt]                " + name);
        return null;
    }
    /// &lt;summary&gt;    Sprite   Sliced        /    /            &lt;/summary&gt;
    private static Sprite LoadPlain(string name)
    {
        var src = Resources.Load&lt;Sprite&gt;(name);
        if (src != null) return src;
        var tex = Resources.Load&lt;Texture2D&gt;(name);
        if (tex != null)
```

```
            {
                var plain = Sprite.Create(tex, new Rect(0f, 0f, tex.width, tex.height),
                    new Vector2(0.5f, 0.5f), UiPixelsPerUnit());
                plain.name = name;
                return plain;
            }
            Debug.LogWarning("[UIArt]               " + name);
            return null;
        }
        private static Sprite CreateVariant(Texture2D tex, Rect rect, string name, Vector4 importedBorder)
        {
            if (tex == null || rect.width &lt; 4f || rect.height &lt; 4f)
            {
                Debug.LogWarning("[UIArt]                  " + name);
                return null;
            }
            // O-1610 slice-b     border spriteBorder       =
            //   64/76/88         =15%      ui_btn_pad
            Vector4 border;
            if (importedBorder.sqrMagnitude &gt; 0f)
            {
                border = importedBorder;
            }
            else
            {
                float bx = Mathf.RoundToInt(rect.width * BorderRatio);
                float by = Mathf.RoundToInt(rect.height * BorderRatio);
                border = new Vector4(bx, by, bx, by);   // border =
            }
            var variant = Sprite.Create(
                tex, rect,
                new Vector2(rect.width * 0.5f, rect.height * 0.5f),   // UI Image   RectTransform    pivot
                UiPixelsPerUnit(),
                0, SpriteMeshType.FullRect,
                border);
            variant.name = name;
            return variant;
        }
        private static float UiPixelsPerUnit()
        {
            // Sliced   border      = borderPx / (spritePPU / canvas.referencePixelsPerUnit)
            //      PPU=512         512px=1       UI       border    5.12×
            //  77px→15                canvas.referencePixelsPerUnit   border 1px→1
            Canvas canvas = UIRoot.Canvas;
            return canvas != null ? canvas.referencePixelsPerUnit : 100f;
        }
#if UNITY_EDITOR
        [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
        private static void EditorSmoke()
        {
```

```
            //           =ScreenRouter.EditorBootTest             OnGUI
            // G10        F                W7       Play
            Debug.Log("[UIArt] ui_btn_pad=" + Describe(ButtonPad) + " / ui_panel_round=" + Describe(PanelRound));
        }
        private static string Describe(Sprite sprite)
        {
            return sprite == null
                ? "  (    )"
                : sprite.texture.width + "x" + sprite.texture.height + " border=" + sprite.border.x.ToString("0");
        }
#endif
    }
}
// UIRoot.cs
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.UI;
namespace GimmeAll.UI
{
    /// &lt;summary&gt;
    ///     UI      BeforeSceneLoad   Canvas/CanvasScaler/EventSystem/SafeArea
    ///        Image   sprite                    ScreenRouter
    /// &lt;/summary&gt;
    public static class UIRoot
    {
        public const float ReferenceWidth = 1080f;
        public const float ReferenceHeight = 1920f;
        /// &lt;summary&gt;   Banner                 SafeArea                 &lt;/summary&gt;
        public const float BannerReserve = 132f;
        //                      Wave 3   G
        public static readonly Color Cream = new Color(1.000f, 0.965f, 0.905f);
        public static readonly Color White = new Color(1.000f, 1.000f, 0.980f);
        public static readonly Color Ink = new Color(0.345f, 0.245f, 0.200f);
        // U320      CR     InkSoft     White 4.42   0.08       4.5  ——
        // RGB(128,102,89)   White 5.29 /   Cream 4.95     8
        public static readonly Color InkSoft = new Color(0.502f, 0.400f, 0.349f);
        public static readonly Color Coral = new Color(1.000f, 0.550f, 0.360f);
        public static readonly Color CoralSoft = new Color(1.000f, 0.720f, 0.520f);
        // U320            #B4611F——        Coral   Cream   2.14= 2.x
        //      Cream ≈4.7         /  /
        public static readonly Color CoralDeep = new Color(0.706f, 0.380f, 0.122f);
        public static readonly Color Lilac = new Color(0.940f, 0.720f, 0.860f);
        // X2334 GUI      CEO                            #C9B0E6——
        //      =CoralSoft tint      =         ·              v2+LilacDeep
        // =   /              ·Ink    ≈4.9    Lilac        /  /
        // X2348 U360Sweep 02        0.788/0.690/0.902     (185,168,210)=     =
        //          ——        #B899DE        ·     =   ·Lilac
        public static readonly Color LilacDeep = new Color(0.722f, 0.600f, 0.871f);
        // X2344 r2 menu tweak 1: gold hue +6.5 deg toward lemon (frozen prep value) — menu gold reads
        // more lemon-yellow, less orange; Ink-on-Gold contrast stays above the 4.5 line (was 5.60).
```

```
        public static readonly Color Gold = new Color(1.000f, 0.809f, 0.180f);
        public static readonly Color Backdrop = new Color(0f, 0f, 0f, 0.55f);
        public static readonly Color ResultVeil = new Color(0.990f, 0.950f, 0.900f, 1f);   // U360-L4C 0.96→0.98→1.0
        public static Canvas Canvas { get; private set; }
        public static RectTransform SafeArea { get; private set; }
        private static Font _font;
        private static Font _displayFont;   // U316/S5          ·OFL         /  /
        private static RectTransform _pendingShadowRt;   // U320 CreateButton            - -
        [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.SubsystemRegistration)]
        private static void ResetStatics()
        {
            // G11 Fast-Enter                              UIRoot
            Canvas = null;
            SafeArea = null;
            _font = null;
            _displayFont = null;   // U316/S5 G11
        }
        [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
        public static void EnsureRoot()
        {
            if (Canvas != null) return; //
#if UNITY_STANDALONE_WIN &amp;amp;&amp;amp; !UNITY_EDITOR
            // U316 CEO      ⑤           fullscreenMode=1
            // CanvasScaler match0.5    1080×1920            ——UI
            //                   =   9:16     =  -100· = ×9/16
            //              =   1080×1920
            var desk = Screen.currentResolution;
            int winH = Mathf.Max(720, desk.height - 100);
            Screen.SetResolution(winH * 9 / 16, winH, false);
#endif
            var root = new GameObject("UIRoot");
            Object.DontDestroyOnLoad(root);
            Canvas = root.AddComponent&lt;Canvas&gt;();
            Canvas.renderMode = RenderMode.ScreenSpaceOverlay;
            var scaler = root.AddComponent&lt;CanvasScaler&gt;();
            scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
            scaler.referenceResolution = new Vector2(ReferenceWidth, ReferenceHeight);
            scaler.matchWidthOrHeight = 0.5f;
            root.AddComponent&lt;GraphicRaycaster&gt;();
            var safe = new GameObject("SafeArea", typeof(RectTransform));
            SafeArea = (RectTransform)safe.transform;
            SafeArea.SetParent(root.transform, false);
            AnchorStretch(SafeArea);
            safe.AddComponent&lt;SafeAreaFitter&gt;();
            var eventSystem = new GameObject("EventSystem");
            eventSystem.transform.SetParent(root.transform, false);
            eventSystem.AddComponent&lt;EventSystem&gt;();
            eventSystem.AddComponent&lt;StandaloneInputModule&gt;(); // legacy Input
        }
        /// &lt;summary&gt;    legacy UI Text      TMP                    &lt;/summary&gt;
```

```
public static Text CreateText(Transform parent, string content, int fontSize, Color? color = null)
{
    var go = new GameObject("Text", typeof(RectTransform));
    var rt = (RectTransform)go.transform;
    rt.SetParent(parent, false);
    AnchorStretch(rt);
    var t = go.AddComponent&lt;Text&gt;();
    t.font = GetFont();
    t.text = content;
    t.fontSize = fontSize;
    t.color = color ?? Ink;
    t.alignment = TextAnchor.MiddleCenter;
    t.horizontalOverflow = HorizontalWrapMode.Wrap;
    t.verticalOverflow = VerticalWrapMode.Overflow;
    t.raycastTarget = false; //
    return t;
}
/// &lt;summary&gt;     ui_btn_pad               tint
/// W13-UI    icon §   B       ——   ≥400px
/// icon   /  null=                ButtonSquash        §   C
/// U320         fg White→Ink White   Coral CR 2.28        3:1  ——Ink
/// Coral 4.24/CoralSoft 5.74/Lilac 5.80/Gold 5.60         fg
/// U320 shadow=true CTA                →      ——
///       btn_pad Sliced·Ink 22%α·   10     ·ButtonShadowPad
///    opt-in             Image             &lt;/summary&gt;
public static Button CreateButton(Transform parent, string label, Vector2 size, Color? bg = null, Color? fg = nu
{
    if (shadow)
    {
        var shGo = new GameObject("BtnShadow", typeof(RectTransform));
        var shRt = (RectTransform)shGo.transform;
        shRt.SetParent(parent, false);
        AnchorCenter(shRt, new Vector2(0f, -10f), size);
        var shImg = shGo.AddComponent&lt;Image&gt;();
        shImg.color = new Color(0.135f, 0.095f, 0.078f, 0.22f);
        var shPad = UIArt.ButtonPad;
        if (shPad != null) { shImg.sprite = shPad; shImg.type = Image.Type.Sliced; }
        shImg.raycastTarget = false;
        _pendingShadowRt = shRt;   //     CreateButton
    }
    var go = new GameObject("Button", typeof(RectTransform));
    var rt = (RectTransform)go.transform;
    rt.SetParent(parent, false);
    AnchorCenter(rt, Vector2.zero, size);
    var image = go.AddComponent&lt;Image&gt;();
    image.color = bg ?? Coral;
    var pad = UIArt.ButtonPad;   //            null →    W2-F
    if (pad != null)
    {
        image.sprite = pad;
```

```
            image.type = Image.Type.Sliced;
        }
        var button = go.AddComponent&lt;Button&gt;();
        button.targetGraphic = image;
        go.AddComponent&lt;ButtonSquash&gt;();   // W13-UI        SneezeButton
        if (shadow &amp;&amp; _pendingShadowRt != null)
        {
            //   =                    RectTransform——
            var tracker = _pendingShadowRt.gameObject.AddComponent&lt;ButtonShadowPad&gt;();
            tracker.Target = rt;
            _pendingShadowRt = null;
        }
        if (icon != null)
        {
            var iconGo = new GameObject("Icon", typeof(RectTransform));
            var iconRt = (RectTransform)iconGo.transform;
            iconRt.SetParent(rt, false);
            var iconImg = iconGo.AddComponent&lt;Image&gt;();
            iconImg.sprite = icon;
            iconImg.raycastTarget = false;   //            Button
            var text = CreateText(rt, label, 56, fg ?? Ink);
            text.font = DisplayFont;   // B7-c/X1439 icon                ——icon                U278
            text.horizontalOverflow = HorizontalWrapMode.Overflow;
            if (size.x &gt;= 400f)
            {
                //                  20px
                float iconSize = Mathf.Clamp(size.y * 0.55f, 64f, 120f);
                AnchorCenter(iconRt, new Vector2(-size.x * 0.5f + 20f + iconSize * 0.5f, 0f), new Vector2(iconSize,
                AnchorCenter(text.rectTransform, new Vector2(iconSize * 0.5f + 12f, 0f), size);
            }
            else
            {
                //      /
                float iconSize = Mathf.Min(120f, size.x * 0.42f, size.y * 0.5f);
                AnchorCenter(iconRt, new Vector2(0f, size.y * 0.16f), new Vector2(iconSize, iconSize));
                AnchorCenter(text.rectTransform, new Vector2(0f, -size.y * 0.26f), size);
            }
            return button;
        }
        var textDefault = CreateText(rt, label, 56, fg ?? Ink);
        textDefault.font = DisplayFont;   // U316/S5     =         ·  =
        textDefault.horizontalOverflow = HorizontalWrapMode.Overflow;
        return button;
    }
    /// &lt;summary&gt;U320 CTA        U327                  -10×scaleFactor
    ///      =       ——Overlay                    ≈0.03   /
    ///    ≈0.01 →
    ///              anchoredPosition          =
    ///   =                  /  pivot/    anchoredPosition    10
    /// ——    /     /   /                        &lt;/summary&gt;
```

```
internal sealed class ButtonShadowPad : MonoBehaviour
{
    internal RectTransform Target;
    private void LateUpdate()
    {
        if (Target == null) return;
        var rt = (RectTransform)transform;
        rt.anchorMin = Target.anchorMin;
        rt.anchorMax = Target.anchorMax;
        rt.pivot = Target.pivot;
        rt.sizeDelta = Target.sizeDelta;
        rt.anchoredPosition = Target.anchoredPosition - new Vector2(0f, 10f);
    }
}
/// &lt;summary&gt;          UIArt.PanelRound              tint    /
///                                              &lt;/summary&gt;
public static Image CreatePanel(Transform parent, Color color, Vector2? anchoredPosition = null, Vector2? size =
{
    var go = new GameObject("Panel", typeof(RectTransform));
    var rt = (RectTransform)go.transform;
    rt.SetParent(parent, false);
    if (anchoredPosition.HasValue &amp;&amp; size.HasValue)
    {
        AnchorCenter(rt, anchoredPosition.Value, size.Value);
    }
    else
    {
        AnchorStretch(rt);
    }
    var image = go.AddComponent&lt;Image&gt;();
    image.color = color;
    if (baseSprite != null)
    {
        image.sprite = baseSprite;
        image.type = Image.Type.Sliced;
    }
    return image;
}
// —— /         SafeArea         ——
public static void AnchorStretch(RectTransform rt)
{
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.pivot = new Vector2(0.5f, 0.5f);
    rt.offsetMin = Vector2.zero;
    rt.offsetMax = Vector2.zero;
}
public static void AnchorCenter(RectTransform rt, Vector2 anchoredPosition, Vector2 size)
{
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
```

```
        rt.pivot = new Vector2(0.5f, 0.5f);
        rt.sizeDelta = size;
        rt.anchoredPosition = anchoredPosition;
    }
    public static void AnchorTop(RectTransform rt, float distanceFromTop, Vector2 size, float xOffset = 0f)
    {
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 1f);
        rt.pivot = new Vector2(0.5f, 1f);
        rt.sizeDelta = size;
        rt.anchoredPosition = new Vector2(xOffset, -distanceFromTop);
    }
    public static void AnchorBottom(RectTransform rt, float distanceFromBottom, Vector2 size, float xOffset = 0f)
    {
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0f);
        rt.pivot = new Vector2(0.5f, 0f);
        rt.sizeDelta = size;
        rt.anchoredPosition = new Vector2(xOffset, distanceFromBottom);
    }
    private static Font GetFont()
    {
        // W4-H    WASM     OS                 W4-D/W4-B2(R)
        //            =
        // Assets/Resources/Fonts/CNFont.ttf Noto Sans CJK SC    SIL OFL 1.1 2.1MB
        //          /           ——              TestFont
        if (_font == null)
        {
            _font = Resources.Load&lt;Font&gt;("Fonts/CNFont");
            if (_font == null) _font = TryBuiltin("LegacyRuntime.ttf");
            if (_font == null) _font = TryBuiltin("Arial.ttf");
            if (_font == null) _font = Font.CreateDynamicFontFromOSFont("Microsoft YaHei UI", 28);
        }
        return _font;
    }
    /// &lt;summary&gt;U316/S5            U278           ZCOOLKuaiLe·OFL     ·
    /// Assets/Resources/Fonts/ZCOOLKuaiLe.ttf ——  /  /
    ///  CNFont         =    LICENSE-    .md &lt;/summary&gt;
    public static Font DisplayFont
    {
        get
        {
            if (_displayFont == null)
            {
                _displayFont = Resources.Load&lt;Font&gt;("Fonts/ZCOOLKuaiLe");
                if (_displayFont == null) _displayFont = GetFont();   //         TestFont      =B7
            }
            return _displayFont;
        }
    }
    private static Font TryBuiltin(string name)
    {
```

```
            try { return Resources.GetBuiltinResource&lt;Font&gt;(name); }
            catch { return null; }   //                 null——
        }
    }
}
// UiBob.cs
using UnityEngine;
namespace GimmeAll.UI
{
    /// &lt;summary&gt;
    /// W21-UX      §   G              ±5px   ——  /
    ///    =    .GetInstanceID        rng     unscaledDeltaTime
    /// Awake     ——      Anchor             AddComponent
    /// &lt;/summary&gt;
    public sealed class UiBob : MonoBehaviour
    {
        public float Amplitude = 5f;     //            px
        public float Period = 2.6f;      //             =
        private Vector2 _base;
        private float _phase;
        private void Awake()
        {
            _base = ((RectTransform)transform).anchoredPosition;
            _phase = (GetInstanceID() % 97) * 0.0645f;   //
        }
        private void Update()
        {
            _phase += Time.unscaledDeltaTime * (6.2831853f / Period);
            var rt = (RectTransform)transform;
            rt.anchoredPosition = new Vector2(_base.x, _base.y + Mathf.Sin(_phase) * Amplitude);
        }
    }
}
// UiDrift.cs
using UnityEngine;
namespace GimmeAll.UI
{
    /// &lt;summary&gt;
    ///      ·     U305 CEO    § .6           20px
    ///               ±10px    20px·16s     ——
    ///                  sizeDelta   +24     12px
    /// unscaled          Awake            ·UiBob
    /// &lt;/summary&gt;
    public sealed class UiDrift : MonoBehaviour
    {
        public float Amplitude = 10f;    //          =2×Amplitude=CEO 20px
        public float Period = 16f;       //         =
        private Vector2 _base;
        private float _phase;
        private void Awake()
```

```
        {
            _base = ((RectTransform)transform).anchoredPosition;
            _phase = (GetInstanceID() % 71) * 0.13f;   //
        }
        private void Update()
        {
            _phase += Time.unscaledDeltaTime * (6.2831853f / Period);
            var rt = (RectTransform)transform;
            // U351/A-P2-6 y        _base.y——    U305 § .6    x     y
            //      MainMenuScreen.ApplyPaperEdge      y
            //  X1958
            rt.anchoredPosition = new Vector2(_base.x + Mathf.Sin(_phase) * Amplitude, rt.anchoredPosition.y);
        }
    }
}
// UiEasing.cs
using UnityEngine;
namespace GimmeAll.UI
{
    /// &lt;summary&gt;
    /// UI       U305          UI           ——  Gameplay
    /// JuiceCurves                 UI     Game/Gameplay
    ///                TestProductFx   /  /
    ///     =CEO    U305 EaseOut=     Pop=EaseOutBack
    /// Cubic=          FlashAlpha=    - -
    /// &lt;/summary&gt;
    public static class UiEasing
    {
        /// &lt;summary&gt;EaseOut    k [0,1] → 0→1           0/1  &lt;/summary&gt;
        public static float EaseOut01(float k)
        {
            if (k &lt; 0f) k = 0f;
            if (k &gt; 1f) k = 1f;
            float inv = 1f - k;
            return 1f - inv * inv;
        }
        /// &lt;summary&gt;EaseOutCubic       0.3s      ·CEO              &lt;/summary&gt;
        public static float EaseOutCubic01(float k)
        {
            if (k &lt; 0f) k = 0f;
            if (k &gt; 1f) k = 1f;
            float inv = 1f - k;
            return 1f - inv * inv * inv;
        }
        /// &lt;summary&gt;EaseIn                       &lt;/summary&gt;
        public static float EaseIn01(float k)
        {
            if (k &lt; 0f) k = 0f;
            if (k &gt; 1f) k = 1f;
            return k * k;
```

```
        }
        /// &lt;summary&gt;Pop EaseOutBack  ·      0→1       1     1.10       1
        /// ——   0.8→1     0.95→1   Q        JuiceCurves.SpawnPop01           &lt;/summary&gt;
        public static float Pop01(float k)
        {
            if (k &lt; 0f) k = 0f;
            if (k &gt; 1f) k = 1f;
            // easeOutBack      s=1.70158  ≈1.10    0/1
            const float s = 1.70158f;
            float t = k - 1f;
            return 1f + (s + 1f) * t * t * t + s * t * t;
        }
        /// &lt;summary&gt;     U305      0→       20% →     →    0
        /// hold=      fall=        unscaled         &lt;/summary&gt;
        public static float FlashAlphaAt(float t, float rise, float hold, float fall, float peak)
        {
            if (t &lt; 0f) return 0f;
            if (t &lt; rise) return peak * EaseOut01(t / rise);
            if (t &lt; rise + hold) return peak;
            float k = (t - rise - hold) / fall;
            if (k &gt;= 1f) return 0f;
            return peak * (1f - EaseIn01(k));
        }
    }
}
// UiFade.cs
using UnityEngine;
namespace GimmeAll.UI
{
    /// &lt;summary&gt;
    /// W21-UX       §   G       =
    /// CanvasGroup   ——   0.18s easeOut          0.12s    SetActive(false)
    ///  OnDisable             PlayLoopDriver      onClick
    ///             Show=    alpha            Show    =no-op
    ///           SetActive(false)           ——MainMenu/HUD
    /// CanvasGroup      Play   alpha=0    ——          =     fail-visible
    ///      /   slideTarget=   RectTransform       46px
    ///      active     alpha        SetActive      _content
    /// unscaledDeltaTime      /     timescale
    /// &lt;/summary&gt;
    public sealed class UiFade : MonoBehaviour
    {
        private const float InTime = 0.18f;
        private const float OutTime = 0.12f;
        private const float SlidePx = 46f;   //
        private const float PopScaleFrom = 0.8f;    // U305        CEO    0.8→1
        private const float PopTime = 0.25f;        // U305    250ms EaseOutBack
        private const float ScreenScaleFrom = 0.95f; // U305        CEO    0.95→1
        private const float ScreenScaleTime = 0.18f; //
        private CanvasGroup _group;
```

```
private RectTransform _slide;      //      null=
private Vector2 _slideBase;
private bool _animIn, _animOut;
private float _t;
private float _fromAlpha;
private RectTransform _scaleRt;    // U305   /         =  ·  =
private float _scaleFrom, _scaleTo, _scaleTime;
private bool _scaling, _reverse;   //        alpha   Pop 0.25s &gt;    0.18s
/// &lt;summary&gt;           slide=            null=     &lt;/summary&gt;
public static UiFade Ensure(GameObject content, RectTransform slide = null)
{
    var f = content.GetComponent&lt;UiFade&gt;();
    if (f == null) f = content.AddComponent&lt;UiFade&gt;();
    if (f._slide == null &amp;&amp; slide != null)
    {
        f._slide = slide;
        f._slideBase = slide.anchoredPosition;
    }
    return f;
}
public void PlayIn()
{
    Init();
    if (!_animOut &amp;&amp; _group.alpha &gt; 0.999f) return;   //         =   no-op
    _fromAlpha = _group.alpha;
    _animIn = true;
    _animOut = false;
    _t = 0f;
    // U305         =0.8→1 Pop 250ms   =0.95→1 180ms+   30ms
    _scaleRt = _slide != null ? _slide : (RectTransform)transform;
    _scaleFrom = _slide != null ? PopScaleFrom : ScreenScaleFrom;
    _scaleTo = 1f;
    _scaleTime = _slide != null ? PopTime : ScreenScaleTime;
    _scaling = true;
    _reverse = false;
    _st = 0f;
    _scaleRt.localScale = new Vector3(_scaleFrom, _scaleFrom, 1f);   //
    if (_slide == null) UiFlash.Flash();
}
private float _st;   //
public void PlayOut()
{
    Init();
    _fromAlpha = _group.alpha;
    _animIn = false;
    _animOut = true;
    _t = 0f;
    // U305         CEO                 1→0.8
    if (_slide != null)
    {
```

```
            _scaleRt = _slide;
            _scaleFrom = 1f;
            _scaleTo = PopScaleFrom;
            _scaleTime = OutTime;
            _scaling = true;
            _reverse = true;
            _st = 0f;
        }
        else
        {
            _scaling = false;
        }
    }
    private void Init()
    {
        if (_group == null)
        {
            _group = gameObject.AddComponent&lt;CanvasGroup&gt;();
            _group.alpha = 0f;   //          Show   0
        }
    }
    private void Update()
    {
        if (_group == null) return;
        // U305          ·unscaled    =Pop            =EaseIn
        if (_scaling &amp;&amp; _scaleRt != null)
        {
            _st += Time.unscaledDeltaTime;
            float sk = Mathf.Clamp01(_st / _scaleTime);
            float curve = _reverse ? UiEasing.EaseIn01(sk) : UiEasing.Pop01(sk);
            float s = Mathf.Lerp(_scaleFrom, _scaleTo, curve);
            _scaleRt.localScale = new Vector3(s, s, 1f);
            if (sk &gt;= 1f)
            {
                _scaling = false;
                _scaleRt.localScale = new Vector3(_scaleTo, _scaleTo, 1f);
            }
        }
        if (_animIn)
        {
            _t += Time.unscaledDeltaTime;
            float k = Mathf.Clamp01(_t / InTime);
            k = 1f - (1f - k) * (1f - k);   // easeOut
            _group.alpha = Mathf.Lerp(_fromAlpha, 1f, k);
            if (_slide != null)
                _slide.anchoredPosition = new Vector2(_slideBase.x, _slideBase.y - SlidePx * (1f - k));
            if (k &gt;= 1f)
            {
                _animIn = false;
                _group.alpha = 1f;
```

```
                    if (_slide != null) _slide.anchoredPosition = _slideBase;
                }
            }
            else if (_animOut)
            {
                _t += Time.unscaledDeltaTime;
                float k = Mathf.Clamp01(_t / OutTime);
                _group.alpha = Mathf.Lerp(_fromAlpha, 0f, k);
                if (_slide != null)
                    _slide.anchoredPosition = new Vector2(_slideBase.x, _slideBase.y + SlidePx * k);
                if (k &gt;= 1f)
                {
                    _animOut = false;
                    _group.alpha = 0f;   //         Show   0
                    if (_slide != null) _slide.anchoredPosition = _slideBase;
                    gameObject.SetActive(false);
                }
            }
        }
    }
}
// UiFlash.cs
using UnityEngine;
using UnityEngine.UI;
namespace GimmeAll.UI
{
    /// &lt;summary&gt;
    ///      U305 CEO    § .3            30ms
    /// UIRoot           Image raycastTarget=false
    ///          0.9 30ms   +90ms   ——
    /// unscaledDeltaTime             UIRoot    SubsystemRegistration
    ///     CurrentAlpha/LastPeakAlpha   FxGate
    /// &lt;/summary&gt;
    public sealed class UiFlash : MonoBehaviour
    {
        private const float RiseTime = 0.015f;   //      1
        private const float HoldTime = 0.030f;   // CEO       30ms
        private const float FallTime = 0.090f;   //
        private const float PeakAlpha = 0.90f;   //
        private static UiFlash _instance;
        private Image _image;
        private float _t;
        private bool _flashing;
        private bool _peaked;   //      30ms        &gt;45ms/             ——
        // U360-S3       Flash        =     ——
        private float _peak = PeakAlpha;
        private float _rise = RiseTime;
        private float _hold = HoldTime;
        private float _fall = FallTime;
        /// &lt;summary&gt;               =0  &lt;/summary&gt;
```

```
public static float CurrentAlpha =&gt; _instance != null &amp;&amp; _instance._image != null
    ? _instance._image.color.a : 0f;
/// &lt;summary&gt;                              &lt;/summary&gt;
public static float LastPeakAlpha { get; private set; }
/// &lt;summary&gt;         FxLoopDriver           public=          &lt;/summary&gt;
public static void ResetProbe() { LastPeakAlpha = 0f; }
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.SubsystemRegistration)]
private static void ResetStatics()
{
    _instance = null;   // Fast-Enter        UIRoot
    LastPeakAlpha = 0f;
}
/// &lt;summary&gt;                        =               &lt;/summary&gt;
public static void Flash()
{
    Flash(PeakAlpha, RiseTime, HoldTime, FallTime);
}
/// &lt;summary&gt;
/// U360-S3                    =   0.18/   0.12s·
/// FeelPulse                    01
/// &lt;/summary&gt;
public static void Flash(float peakAlpha, float rise, float hold, float fall)
{
    if (_instance == null)
    {
        var go = new GameObject("UiFlash", typeof(RectTransform));
        var rt = (RectTransform)go.transform;
        rt.SetParent(UIRoot.Canvas.transform, false);
        UIRoot.AnchorStretch(rt);
        var img = go.AddComponent&lt;Image&gt;();
        img.color = new Color(1f, 1f, 0.98f, 0f);   // UIRoot.White   ·
        img.raycastTarget = false;
        _instance = go.AddComponent&lt;UiFlash&gt;();
        _instance._image = img;
    }
    _instance.transform.SetAsLastSibling();   //           /
    _instance._flashing = true;
    _instance._peaked = false;
    _instance._t = 0f;
    _instance._peak = Mathf.Clamp01(peakAlpha);
    _instance._rise = Mathf.Max(0f, rise);
    _instance._hold = Mathf.Max(0f, hold);
    _instance._fall = Mathf.Max(0f, fall);
}
private void Update()
{
    if (!_flashing) return;
    _t += Time.unscaledDeltaTime;
    float a;
    if (!_peaked)
```

```
            {
                _peaked = true;
                a = _peak;   //                  30ms      ·CEO
            }
            else
            {
                a = UiEasing.FlashAlphaAt(_t, _rise, _hold, _fall, _peak);
            }
            if (a &gt; LastPeakAlpha) LastPeakAlpha = a;
            if (_image != null) _image.color = new Color(1f, 1f, 0.98f, a);
            if (_t &gt;= _rise + _hold + _fall)
            {
                _flashing = false;
                if (_image != null) _image.color = new Color(1f, 1f, 0.98f, 0f);
            }
        }
    }
}
// WardrobeScreen.cs
using UnityEngine;
using UnityEngine.UI;
using GimmeAll.Core;
namespace GimmeAll.UI
{
    /// &lt;summary&gt;
    /// W22-META     §      ·    18      =    p≥5k
    ///    tint 10% OutfitEquipped     ——Gameplay         ChapterPalette
    ///      API CollectionScreen            19     +18
    ///     =vh.progress    CodexCatalog   ——UI     Levels
    ///    =vh.wardrobe int   0=        =AfterSceneLoad     OutfitEquipped
    ///        ——ActorDudu    pending
    /// &lt;/summary&gt;
    public sealed class WardrobeScreen : MonoBehaviour
    {
        private const string EquipKey = "vh.wardrobe";
        private const float PanelW = 940f, PanelH = 1660f;
        private static WardrobeScreen _instance;
        // §    A       canon        =   1-18
        private static readonly string[] OutfitNames =
        { "", "     ", "     ", "     ", "     ", "     ", "     ", "     ", "     ",
          "    ", "    ", "    ", "    ", "    ", "    ", "    ", "    ", "    ",
          "    ", "    ", "    " };
        public static void Show()
        {
            if (_instance == null)
            {
                var go = new GameObject("WardrobeScreen", typeof(RectTransform));
                ((RectTransform)go.transform).SetParent(UIRoot.SafeArea, false);
                UIRoot.AnchorStretch((RectTransform)go.transform);
                _instance = go.AddComponent&lt;WardrobeScreen&gt;();
```

```
    }
    _instance.Open();
}
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
private static void PublishEquippedAtBoot()
{
    PublishEquip(PlayerPrefs.GetInt(EquipKey, 0));   //          →ActorDudu    pending
}
private RectTransform _content, _panel;
private Image[] _slotLock = new Image[19];   // CEO     10-09        →   =
private Button[] _slotBtn = new Button[19];
private void Awake()
{
    //    Backdrop +
    _content = UIRoot.CreatePanel(transform, UIRoot.Backdrop).rectTransform;
    _panel = UIRoot.CreatePanel(_content, UIRoot.White, new Vector2(0f, 0f), new Vector2(PanelW, PanelH), UIArt.
    var title = UIRoot.CreateText(_panel, "     ", 56, UIRoot.Ink);
    title.font = UIRoot.DisplayFont;   // X1436/B7: wardrobe panel heading = display-font track (U278)
    UIRoot.AnchorTop(title.rectTransform, 34f, new Vector2(700f, 90f));
    // 19       0=   + 1-18       ×10      128    12——
    for (int i = 0; i &lt; 19; i++)
    {
        int idx = i;
        int col = i % 2, row = i / 2;
        var btn = UIRoot.CreateButton(_panel, SlotName(i), new Vector2(430f, 128f), UIRoot.CoralSoft, UIRoot.Ink
        UIRoot.AnchorTop((RectTransform)btn.transform, 130f + row * 140f, new Vector2(430f, 128f), col == 0 ? -2
        // CEO     10-09            …           =
        //          =Coral/   =CoralSoft/  =Lilac+   ——
        var lockGo = new GameObject("LockIcon", typeof(RectTransform));
        var lockRt = (RectTransform)lockGo.transform;
        lockRt.SetParent(btn.transform, false);
        UIRoot.AnchorCenter(lockRt, new Vector2(175f, 36f), new Vector2(40f, 40f));
        var lockImg = lockGo.AddComponent&lt;Image&gt;();
        lockImg.sprite = UIArt.Icon("lock");
        lockImg.raycastTarget = false;   //        U277
        _slotBtn[i] = btn;
        _slotLock[i] = lockImg;
        btn.onClick.AddListener(() =&gt; OnSlotClicked(idx));
    }
    var close = UIRoot.CreateButton(_panel, "   ", new Vector2(430f, 120f), UIRoot.Gold, UIRoot.Ink);
    UIRoot.AnchorBottom((RectTransform)close.transform, 40f, new Vector2(430f, 120f));
    close.onClick.AddListener(Close);
    _content.gameObject.SetActive(false);   //        W21-UX
}
private static string SlotName(int i) { return i == 0 ? "  " : OutfitNames[i]; }   // CEO     10-09      ·
/// &lt;summary&gt;      §           k   =     vh.progress ≥ 5k CodexCatalog       &lt;/summary&gt;
public static bool IsUnlocked(int chapter)
{
    if (chapter &lt;= 0) return true;   //
    int p = PlayerPrefs.GetInt(_progressKey, 0);
```

```
            return p &gt;= chapter * 5;
        }
        private const string LevelProgressKey = "vh.progress";   // LevelProgressStore.DefaultPrefsKey    UI     Levels
        private static string _progressKey = LevelProgressKey;
        /// &lt;summary&gt;      LevelProgressStore               DeleteKey    &lt;/summary&gt;
        public static void UseProgressKeyForTests(string key)
        {
            _progressKey = string.IsNullOrEmpty(key) ? LevelProgressKey : key;
        }
        private void Open()
        {
            Refresh();
            _content.gameObject.SetActive(true);
            UiFade.Ensure(_content.gameObject, _panel).PlayIn();   // W21-UX
        }
        private void Close()
        {
            UiFade.Ensure(_content.gameObject, _panel).PlayOut();
        }
        private void Refresh()
        {
            int equipped = PlayerPrefs.GetInt(EquipKey, 0);
            for (int i = 0; i &lt; 19; i++)
            {
                bool unlocked = IsUnlocked(i);
                var img = _slotBtn[i].GetComponent&lt;Image&gt;();
                img.color = unlocked ? (i == equipped ? UIRoot.Coral : UIRoot.CoralSoft) : UIRoot.Lilac;
                _slotLock[i].enabled = !unlocked;   //   =
            }
        }
        private void OnSlotClicked(int idx)
        {
            if (!IsUnlocked(idx)) return;   //
            PlayerPrefs.SetInt(EquipKey, idx);
            PlayerPrefs.Save();
            PublishEquip(idx);
            Refresh();
        }
        private static void PublishEquip(int chapter)
        {
            if (chapter &lt;= 0)
            {
                GameEventBus.Publish(new OutfitEquipped { Chapter = 0, R = -1f, G = -1f, B = -1f });
                return;
            }
            Color c = ChapterPalette.ButtonOf(chapter);   //    tint   UI        Gameplay
            GameEventBus.Publish(new OutfitEquipped { Chapter = chapter, R = c.r, G = c.g, B = c.b });
        }
    }
}
```